claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_7049e1ed-b93\agent-ace53c1533b7fda26.jsonl`
- SHA-256: `f50afdc3c2a41348b8ea486add5a5eaa5f0e672475de53cb6b2facd61de97e34`
- Source modified: `2026-07-02T04:58:14+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_7049e1ed-b93`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T04:45:39.422Z)

You are a senior engineer producing ONE isolated PR. Today is 2026-07-02.
REPO: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (a shared checkout ? follow the safety rules exactly).
AUDIT FINDINGS YOU ARE FIXING: R2-8 (retention sweep unscheduled) ? Grep C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-guru_2026-07-02.md for these IDs and read the full evidence + verifier notes before coding.
TASK BRIEF:
In C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer: the T11 sweep (tools/cleanup_caches.py, --uploads-retention-days default 7) exists but nothing runs it. Add: (1) a default-OFF in-server periodic sweep ? config retention.sweep_enabled=false + retention.sweep_interval_hours=24 ? running the uploads-zone sweep logic in a daemon thread/timer on the server boot path (reuse the tool's classify/purge functions by import, do not shell out; log each purge loudly per the observability working agreement); (2) deploy documentation: a systemd timer example in the deploy kit + a note in deploy/cloudrun/README.md Gotchas + the tool's docstring pointing at the knob. Default-OFF ? zero behavior change; the owner flips it on hosted boxes. Tests: timer respects the flag, sweep invocation path unit-tested with a fake clock/tmpdir.

BRANCHING SAFETY (mandatory ? shared checkout, sibling sessions exist):
1. In the MAIN repo checkout run: git fetch origin
2. Create an ISOLATED WORKTREE: git worktree add "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-<shortname>" -b <branch-name> origin/master   (branch explicitly FROM origin/master ? never from the current local branch; use origin/main for repos whose default is main)
3. Do ALL work inside the worktree. Before committing: git branch --show-current + git log --oneline -1 must show YOUR branch on the origin-master base.
4. Stage EXPLICIT paths only (git add <file> <file>) ? NEVER git add -A / . / -u
5. Before opening the PR: git log --oneline origin/master..HEAD must list ONLY your commits.
COMMIT/PR CONVENTIONS:
- Commit message: conventional style matching the repo's history; end the message with a blank line then: Co-Authored-By: Claude Fable 5 <noreply@anthropic.com>
- PR body: what/why, a "## Verification" section stating exactly what you ran locally and what CI covers, and end with: ? Generated with [Claude Code](https://claude.com/claude-code)
- Do NOT merge the PR. Do NOT remove the worktree (a fixer may need it). Return the PR number, branch, worktree path.
QUALITY BAR:
- Read the repo's CLAUDE.md first; for the engine also docs/CODE_MAP.md §0 (file placement) before adding any new file.
- Add/extend tests for every behavior change. Run the targeted tests you touched (python -m pytest tests/<relevant> -q) + ruff check on changed files; if mypy config exists, run mypy on touched modules. Full suite is CI's job ? do not run it.
- Default-safe: new knobs default OFF / behavior-preserving unless the task explicitly says

```
otherwise.
- The full audit evidence is in C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-
Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/AUDIT_REPORT_khelsutra-
guru_2026-07-02.md ? Grep it for your finding IDs and READ the verifier notes; they contain
corrections (exact file:line, scoping traps) you MUST honor.

If the brief's premise turns out false against current code (something already fixed, file
moved), adapt honestly and say so in notes ? never force a pointless diff. If you genuinely
cannot complete (env broken, auth missing), open NO PR and explain in notes with pr_number=0.
```

2. user (2026-07-02T04:45:46.769Z)

```
468-### R2-3 [P2/S] ? sports-data-collector
469-**Collector pins obsreport as a plain PyPI requirement while the name is unclaimed on PyPI ?
dependency-confusion window and install broken today**
470-- verdict: CONFIRMED
471-[Omitted long context line]
472-[Omitted long context line]
473-
474-### R2-4 [P2/S] ? badminton-highlight-indexer
475-**No timeout on any serving-path ffmpeg subprocess ? one hung ffmpeg permanently wedges the
single-worker job queue**
476-- verdict: CONFIRMED
477-[Omitted long context line]
478-[Omitted long context line]
479-
480-### R2-5 [P1/S] ? badminton-highlight-indexer
481-**Engine CLAUDE.md 'Current state' section is ~3 weeks false ? claims platform/owned-model
work 'has not started'**
482-- verdict: CONFIRMED
483-[Omitted long context line]
484-[Omitted long context line]
485-
486-### R2-6 [P1/M] ? sports-data-collector
487-**Golden-corpus registry rot: all 6 owned 'curated' videos point at a deleted absolute
local_path and the 4.9GB source videos are single-copy with no documented backup**
488-- verdict: CONFIRMED
489-[Omitted long context line]
490-[Omitted long context line]
491-
492-### R2-7 [P2/S] ? non-git
493-**Roora vendor pilot record (paid deliverables + corrected golden labels + 3 videos) is a
single unversioned local copy at risk of loss**
494-- verdict: CONFIRMED
495-[Omitted long context line]
496-[Omitted long context line]
497-
498:### R2-8 [P1/S] ? badminton-highlight-indexer
499-**Upload retention sweep tool exists but nothing schedules it ? unbounded upload growth +
unenforced 7-day privacy retention on any hosted deployment (#301)**
500-- verdict: CONFIRMED
501-[Omitted long context line]
502-[Omitted long context line]
503-
504-
505-## Owner-gated (flagged, no action)
506-
507-[Omitted long context line]
508-
509-[Omitted long context line]
510-
511-
512-## Critic additions (round 1 completeness pass)
513-
```

```
514-- [P1/M] critical-fix: **Golden-corpus registry rot: all 6 owned 'curated' videos point at a
deleted absolute path, and the 4.9GB source videos are single-copy with no documented backup**
(`sports-data-collector/data/sports_metadata.db:0`)
515-[Omitted long context line]
516-  - fix: Add a `curate verify` (or extend cleanup.py) that resolves every registered
local_path and flags dangling rows; re-point the 6 golden rows to the current 'khelsutra-
guru/Annotation Setup/Golden Labelled' location; store paths relative to a configurable corpus
root instead of absolute machine paths; and back the 4.9GB Archive up to the existing GCS
bucket, recording the gs:// URI in video_url so the registry has a durable pointer.
517-
518-- [P2/S] ci-infra: **sports-obsreport has no CI at all ? combined with the engine's skipped
reporting tests, zero reporting-stack code is gated anywhere** (`sports-
obsreport/pyproject.toml:7`)
519-[Omitted long context line]
520-  - fix: Add a minimal .github/workflows/ci.yml to sports-obsreport: checkout, pip install
-e .[dev], pytest --cov on push/PR. This closes the library half of the reporting-stack gap
independently of the engine's obsreport-lane work.
521-
522-- [P3/S] hygiene: **Engine .git bloated to 281MB by ~229MB of dangling raw match-video
blobs; 6.5MB yolov8n.pt is permanently in pushed history and .gitignore has no guard against re-
staging raw videos** (`badminton-highlight-indexer/.gitignore:49`)
523-[Omitted long context line]
524-  - fix: Run `git gc --prune=now` in the local checkout to reclaim ~260MB, and add a broad
guard (e.g. `*.mp4` with explicit `!` exceptions for any tracked fixtures, or ignore the video
scratch dirs) to .gitignore. Optionally note yolov8n.pt in docs as accepted history weight;
rewriting pushed history is not worth it at 6.5MB.
525-
526-- [P3/S] tech-debt: **obsreport dependency URL still points at the pre-transfer avidullu org
? installs survive only via GitHub's transfer redirect** (`badminton-highlight-
indexer/pyproject.toml:68`)
527-[Omitted long context line]
528-  - fix: Update the dependency URL in pyproject.toml (and the requirements.txt comment) to
github.com/Khelsutra/sports-obsreport.git and pin the v0.1.3 commit SHA instead of the tag,
matching whatever convention the trackers-pin fix adopts.
```

3. user (2026-07-02T04:45:47.904Z)

```
CLAUDE.md
README.md
__pycache__
artifacts
backend
backend.db
config.json
deploy
docs
eval_baselines
indexer.db
mypy.ini
output
pyproject.toml
pytest.ini
rally_index.db
requirements.txt
ruff.toml
run_all_tests.py
run_eval.py
run_regression.py
scratch
scripts
tests
tools
training
```

4. user (2026-07-02T04:45:55.451Z)

- reasoning: CONFIRMED for the paid deliverables + corrected golden labels; the "3 videos" component of the claim is REFUTED (Drive copies exist). Evidence: (1) Single-copy artifacts verified on disk: raw paid CVAT deliverables at 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup/Archive/Roora Pilot Review - 6 June 2026/' ? badminton/{badminton, badminton (1), badminton (2)}/Annotations/*.xml (~2.3 MB per-frame boxes), pickleball/*.zip (3 zips, 260 KB), tabletennis/*.zip (3 zips, 213 KB) ? plus the human-corrected golden labels 'Annotation Setup/golden_labels_badminton.csv' (2.5 KB, mtime 2026-06-06) and the rescued multisport goldens 'Annotation Setup/Trajectories/roora/{pickleball,tabletennis}_rallies.csv'. (2) Not in git: sports-data-collector tracks only the AS-DELIVERED parse (data/roora_badminton_canonical.csv, verified via git ls-files + diff ? it still has rally 2 = 0:02.503?0:32.533, rally 7 end 1:51.612, no rallies 7.5/16.5), while golden_labels_badminton.csv holds the corrections (rally 2 start?0:29, rally 7 end?1:38, missed rallies 7.5@1:46 and 16.5@3:34 added, winner?out fixes) ? those corrections exist NOWHERE else (find over C:/Users/avidu/Projects + Downloads returned only this one file). (3) Not in GCS: recursive 'gcloud storage ls -r gs://khelsutra-rally-corpus/**' contains zero objects matching roora|pilot|pickle|tabletennis|sample_long|golden_labels; labels/ holds exactly the 15 canonical corpus clips (pilot video not among them); archive/ holds only a droplet archive. (4) Not on Drive: 'G:/My Drive/Sharing' has guidelines/briefs and 'Golden Annotations' (MahadevpuraSingles only) ? no CVAT exports. (5) Outside every backup convention: badminton-highlight-indexer/docs/DATA_IN_GCS.md §5 gap table (lines 117?131) covers only the 15-clip corpus and never mentions the pilot artifacts (and its line 121/142 path 'C:\?\Annotation Setup\Collect\Trajectories\' is stale ? the folder moved, no Collect dir exists); the F: backup (memory checkpoint 2026-06-22) covered only the 7 new-clip labels; docs/archives/roora-vendor/README.md archives only guidelines+ADR, not pilot results; F: is currently unmounted. REFUTED part: the 3 pilot videos have byte-size-identical cloud copies at 'G:/My Drive/Sharing/Annotation Videos/Pilot Videos/' (Sample Long Video - Baadminton.MP4 = 2,882,861,900 B; Video 2 - Pickleball.mp4 = 1,435,276,306 B; Video 3 - Tabletennis.mp4 = 871,944,715 B ? all match local), and the local 'Annotation Videos/desktop.ini' references GoogleDriveFS.exe proving Drive File Stream provenance. Owner's own memory audit had flagged the precursor ("Roora multisport CSVs live ONLY in %TEMP%? one cleanup from loss"); the rescue moved them to Annotation Setup but they remain a single unversioned C:-disk copy.
- notes: Correction to the finding: the at-risk payload is NOT the 4.9 GB folder ? the videos are safe on the owner's Google Drive. What is genuinely one C:-disk failure (or one folder-cleanup) from loss is ~3 MB of paid, partly irreplaceable work product: the raw vendor CVAT exports for all 3 sports (box geometry the tracked canonical CSVs discard), the human frame-by-frame correction record (golden_labels_badminton.csv + rally_review.csv ? hours of review labor and the calibration evidence behind the vendor feedback), the pickleball/tabletennis golden CSVs backing the recorded TT-0.909/PB-0.308 multisport result, and the pilot review MD/PDF record. Vendor re-export from CVAT job 4075945 is not under owner control. Failure scenario: C: SSD failure or a cleanup of the non-git 'Annotation Setup' folder silently destroys the only record of the paid pilot and its corrected labels; nothing in DATA_IN_GCS.md's migration plan would ever pick it up because the pilot artifacts are absent from its §5 gap table. Fix (S, not owner-gated for the metadata): one 'gcloud storage cp -r' of 'Annotation Setup/Archive/Roora Pilot Review - 6 June 2026/' (excluding Annotation Videos/, already on Drive) + 'Annotation Setup/{golden_labels_badminton.csv,rally_review.csv,Trajectories/roora/}' to gs://khelsutra-rally-corpus/archive/roora-pilot-2026-06/ (archive/ prefix precedent already exists; ~3?10 MB, negligible cost), then add a Roora-pilot row to DATA_IN_GCS.md §5 and fix its stale 'Annotation Setup\Collect\Trajectories' path (lines 121/142). Including the 3 pilot videos in the bucket is the only owner-gated piece (DATA_IN_GCS §7 open decision 4, ~5 GB ? $0.10/mo) and is optional given the Drive copies.

R2-8 [P1/S] ? badminton-highlight-indexer

Upload retention sweep tool exists but nothing schedules it ? unbounded upload growth + unenforced 7-day privacy retention on any hosted deployment (#301)
- verdict: CONFIRMED
- reasoning: Tool exists: badminton-highlight-indexer/tools/cleanup_caches.py (T11 retention sweep, --uploads-retention-days default 7.0 at ~line 407; classify() purges uploaded videos + .partial-* in the uploads zone; landed cb7394b/#299). Nothing schedules it: workspace-wide grep for cleanup_caches hits only the tool/tests/docs and an error string (backend/pipeline/detectors/base.py:162); zero references in .github/workflows/nightly.yml, deploy/cloudrun/, deploy/digitalocean/, scripts/, and zero cron/timer/sweep references in khelsutra/deploy/droplet/ (khelsutra-engine.service has no companion timer). Backend startup only sweeps Gemini remote orphans (backend/main.py:1808 _maybe_sweep_gemini_orphans), not local uploads. Issue #301 is OPEN (viewed via gh today); no scheduling commit in git log between cb7394b and master e410136. Deployment is LIVE: khelsutra/deploy/DEPLOY_ALPHA.md line 3 "Status:

LIVE (deploy live-verified on the droplet 2026-06-22)"; uploads land via backend/api/uploads.py into security.upload_dir (/srv/khelsutra/incoming/uploads per runbook §2.2).
- notes: Sharper failure scenario: the alpha droplet is live and accepting uploads behind Cloudflare Access; without any scheduled `python -m tools.cleanup_caches --apply`, a tester's footage uploaded day 0 is still on disk day 30+ (unenforced 7-day privacy retention, an explicit owner policy per the tool docstring, 2026-06-21), and output/uploads grows unboundedly until require_free_bytes (backend/api/uploads.py:93) starts rejecting new uploads (service degradation on the small CPU droplet). Mitigations that keep this P1 not P0: runbook notes testers had not yet exercised the upload CUJ (OTP login + Gemini key rotation pending as of 2026-06-22), and the owner can run the sweep manually. Fix scope (S): do NOT re-implement the sweep ? ship a systemd timer + oneshot unit (or cron line) alongside khelsutra/deploy/droplet/khelsutra-engine.service that runs `python -m tools.cleanup_caches --apply --uploads-retention-days 7 --uploads-dir <security.upload_dir>` daily with --json output captured to journald; default-OFF/opt-in flag so local single-user installs are unaffected; document enable+verify steps in DEPLOY_ALPHA.md; one smoke test asserting the wrapper passes the right flags (tool logic already unit-tested in tests/test_cleanup_caches.py). Issue #301's acceptance criteria already spec this exactly. Partial owner gate: shipping the timer unit + docs is not gated, but *enabling* it on the live droplet is an ops action requiring the owner's SSH access, and #301 flags one open design call (whether apply should also skip uploads referenced by non-terminal jobs rows) ? recommend defaulting to the time-window-only behavior and letting the owner ratify.

Owner-gated (flagged, no action)

5. user (2026-07-02T04:46:02.625Z)

```
1       # CLAUDE.md ? project memory / agent handoff
2
3       Local, AI-powered **badminton (? racket-sports) highlight indexer + rally detector**:
4       FastAPI + SQLite + ffmpeg + GPU CV/AI, with paid **Gemini** as the cloud AI. Sibling
5       repos: `sports-data-collector`, a VLC-based `rally-annotator`.
6
7       ## Read these first
8       - **`docs/README.md`** ? doc index. **`docs/CODE_MAP.md`** ? code navigation + data
9         contracts (the cold-start map; read before touching code).
10      - **Authoritative owned-model roadmap:** `docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md` (+
the
11        live `docs/NEXT_STEPS.md`). These SUPERSEDE the 2026-06-07 strategy docs below where
they
12        disagree (licensing, Gemma-4 stance, base model, conversational-QA).
13      - **Strategy (post-scaffolding):** `docs/COMMERCIALIZATION.md`,
`docs/PMF_MARKET_RESEARCH.md`,
14        `docs/PLATFORM_ARCHITECTURE.md`, `docs/DEFERRED_HARDENING_PLAN.md`. Also
`docs/ROADMAP.md`
15        (offline-segmenter narrative) and `docs/BACKLOG.md`.
16      - **Rally-detection quality track (design merged 2026-06-13, owner-gated, implementation
in progress):**
17        `docs/HOW_RALLY_DETECTION_WORKS.md` (2-layer mental model) ?
`docs/MULTI_SIGNAL_FUSION_PLAN.md`
18        (fuse shuttle + person + audio cues; the held-shuttle bug is the already-built
`rally_gate.py`
19        "Gate A" **not wired into production** ? cheapest win) ? `docs/EXPERIMENT_HARNESS.md`
(A/B + shadow
20        eval so cues land flag-gated/default-OFF with zero regression) ?
`docs/ROORA_LABELING_GUIDELINES.md`
21        (v8 vendor labeling spec). **`docs/NEXT_STEPS.md` has the prioritized pick-up for this
track.**
22      - **History:** `docs/archives/` ? ADRs (`decisions/DECISIONS.md`), research, and
23        **`checkpoints/`** (latest: `2026-06-strategy-foundation/`). **Archive policy:** only
move
24        **terminal-state** (DONE/REJECTED) work; live docs stay at `docs/` top level ? and
**before
25        archiving, HONESTLY update the doc first** (verify claims vs code ? flip status + a
completion
```

```
26        note ? update inbound refs ? keep the lineage ? then `git mv`), per the checklist +
living
27        lifecycle register in **`docs/DOC_STATUS.md`**.
28
29      ## Current state (June 2026)
30      - **Scaffolding complete**; strategy foundation + owned-model roadmap merged. **No
31        platform/owned-model implementation has started.** A 2026-06-10 audit-driven hardening
32        sweep landed (licensing gate, detector keying, re-ingest safety, config, eval gate,
API
33        validation, reliability, CI + test coverage) ? see git history.
34      - **Two tracks, both owner-gated:** the **committed next PLATFORM slice = async job
model**
35        (`DEFERRED_HARDENING_PLAN.md` #16, single-tenant); the **owned-model track** starts at
a
36        greenlit Phase-0 milestone (`OWNED_MODEL_IMPLEMENTATION_PLAN.md`). **Do not start
37        implementation without explicit owner approval.**
38
39      ## Architecture in one breath
40      - **Pluggable registries** (ABC + `Registry` singleton): segmenters / providers /
validators
41        / sports / annotations ? see `backend/pipeline/segmenters/base.py`. *Future:*
42        `StorageBackend` + `ComputeBackend` registries (PLATFORM §3).
43      - `video_id` = MD5 of the file ? `backend/utils/hashing.py::compute_video_id` (one
helper).
44      - Typed config = `backend/config/` (Pydantic). DB schema evolves via the `PRAGMA
45        user_version` migration ladder in `backend/infrastructure/database.py`.
46
47      ## Committed guardrails (non-negotiable)
48      - **Paid LLMs (Gemini/OpenAI/Anthropic) = inference / serving / fallback ONLY ? never a
49        training data source** (terms/copyright). Commercial-clean licensing for EVERY
dependency ?
50        code, data, AND weights: **MIT / Apache-2.0 / BSD only** (ratified 2026-06-09, owner-
agreed).
51      - **Owned-model base:** **Qwen3-VL (Apache-2.0)** primary, InternVL3 (MIT) fallback;
**Gemma-4
52        is AMBER / bench-only** (Apache grant likely but Prohibited-Use posture unconfirmed ?
counsel
53        needed; do NOT build on it without sign-off). **Reject Gemma 3** (viral license).
**Exclude
54        minors** from the training pool; per-user training data needs a separate explicit opt-
in.
55      - **v1 = rally + highlight + history + correction loop**; **gait/biometrics strictly
future**
56        (GDPR Art. 9).
57      - **Generic data schema**; `coach ? athlete` is an **optional overlay**, not baked in.
58      - **Compute = GCP ONLY (ADR-013, 2026-06-20):** GPU training + serving run on **GCP**
(`deploy/gcp/`;
59        Cloud Run+GPU for serving per ADR-012). **DigitalOcean is DROPPED for compute** (DO
GPU not enabled
60        on the account + not cleanly credit-funded; Paperspace is subscription-gated). Don't
re-evaluate
61        DO/Paperspace for compute without a new ADR; the `$200` DO credits go to **non-compute
only**. The
62        `deploy/digitalocean/*` scripts are kept solely as provider-agnostic Linux-GPU setup
reused by GCP.
63
64      ## Workflow conventions
65      - **Branch off `master`; ONE PR per work item; NO PR stacking.** Don't open a PR unless
asked.
66      - Tests: `python run_all_tests.py` (offline, fully mocked). Python 3.10+ (the `trackers`
package requires ?3.10); `ffmpeg` required.
67      - Never push to the default branch directly; descriptive commit messages.
68      - **Substantial design/project work gets a tracked project doc** following
`docs/PROJECT_DOC_TEMPLATE.md` (status header · decisions-locked · progress tracker ·
definition-of-done; archived on completion). Precedents: `docs/RALLY_QUALITY_RESEARCH.md` §7,
```

`docs/GOLDEN_REGRESSION_FIXTURES.md`.
69   - **Launch / "bit-flip" PRs** (a default-flip that drives a tracked project to terminal state) MUST, in the SAME PR, run the `docs/DOC_STATUS.md` §2 archival due-diligence on that project's docs (verify-vs-code ? flip status + completion note ? update inbound refs ? keep ADR lineage ? `git mv` to `docs/archives/` ? record in DOC_STATUS §3), and fix any internal-link breakage the move causes. Enforced by `.github/PULL_REQUEST_TEMPLATE.md` + the `link-check` CI job (`python -m tools.check_md_links`).
70   - ?? Hand the heaviest machine-side checks (real video, the full GPU/WSL/WASB suite) to the owner,
71   who has the golden corpus + labelled videos; verify pure-logic paths locally.
72

6. user (2026-07-02T04:46:04.051Z)

```
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy:
cloudrun
digitalocean
gcp

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy/cloudrun:
Dockerfile
README.md
cloudbuild.yaml

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/tools:
__init__.py
__pycache__
check_md_links.py
cleanup_caches.py
generate_highlights.py
leak_scan.py
```

7. assistant (2026-07-02T04:46:09.674Z)

First, per the repo-freshness convention, let me pull the main checkout.

8. user (2026-07-02T04:46:12.467Z)

master...origin/master

```
Already up to date.
```

9. user (2026-07-02T04:46:17.418Z)

CODE_MAP.md

Purpose: cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.

Regenerate: rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).

? **2026-06-10 audit-hardening sweep changed several behaviors documented below ? trust git history over stale lines until the next full regen:** `add_video` is now an **UPSERT** (no FK-cascade wipe on re-ingest; segmentation is destroy-AFTER-confirm via `segment_watermarks`/`prune_segments`); detector artifacts are keyed by **video_id**, not basename; **`indexing.wasb.*` is now a real typed config** (was silently dropped); `timeout_sec` defaults **1800s** (was 0/hang-forever); SQLite opens with **WAL + busy_timeout**; `match_intervals` requires real overlap (iou>0); `run_regression` baseline lives in tracked `eval_baselines/` and **exits 3 on a missing baseline**; the API validates `video_path`/`output_filename`. New tests: `test_api_endpoints`, `test_compiler`, `test_wasb_runner`, `test_reingest_safety`, `test_detector_keying`, `test_config_wasb`,

`test_path_safety`, `test_reliability_hardening` (+ collector `test_licensing_gate`,
`test_import_guard`, `test_cvat_robustness`).

LEGEND
- `@path` = file (always repo-relative)
- `::sym` = symbol (class/func/const) in the nearest preceding `@path`
- `->` = dataflow / call / "produces"
- `§` = data contract (canonical shape)
- `?` = gotcha / footgun
- `?` = registry / plugin extension point
- `WSL` = runs inside WSL2 conda env, NOT the Windows Python process

---

0. REPO LAYOUT ? where new files go (READ BEFORE committing a new file)

Top-level tree and the **placement rule** for each kind of file. When unsure, prefer an existing
registry / extension point (`?`) over a new top-level file.

You're adding?	Put it in?	Notes
Backend / runtime code	`backend/<subsystem>/`	`pipeline/{segmenters,detectors,validators}`, `providers/`, `sports/`, `annotations/`, `infrastructure/`, `reporting/`, `stitcher/`, `utils/`, `config/`, `features/`, `api/`. Most additions are **register a class in the relevant `?` registry**, not a new module tree.
Eval LIBRARY (importable, pure ? metrics, calibration, a scorer, a feature)	`backend/eval/`	Pure core, no side-effects at import; CLI/I-O lives in a separate driver. These are **imported widely** (e.g. `calibrate_wasb` by 8 sites) ? they are library, not scripts; do NOT move them out.
Standalone run-driver / one-off entry CLI (has `__main__`, NOT imported by app code)	**`scripts/`**	e.g. `scripts/run_phase3_eval.py`, `scripts/run_process_and_stitch.py`. Run as `python scripts/<name>.py`. If it needs `load_dotenv()`/`sys.path` before importing backend, add `"scripts/<name>.py" = ["E402"]` to `ruff.toml`.
Ops / maintenance tool (not part of the served app ? cache GC, batch admin)	`tools/`	Run as `python -m tools.<name>` (it's an importable package). e.g. `tools/cleanup_caches.py`. Keep the destructive decision path pure + unit-tested.
Training code (model fitting)	`training/`	Behind the CI-enforced import wall (ADR-008) ? `backend.main` must not import it. Own `requirements-train.txt`.
A test	`tests/test_*.py`	pytest auto-discovers; the full-suite lane has a **70% coverage floor**. Pure/offline + mocked (no GPU/WSL/network).
A doc (design, plan, living log)	`docs/`	Index it in `docs/README.md`. Only **terminal-state** (DONE/REJECTED) work moves to `docs/archives/`.
A throwaway repro / debug script	`scratch/`	**Gitignored** + excluded from ruff/pytest. Never imported by app code.
A model artifact (weights + manifest)	`artifacts/<name>/<ver>/`	ADR-008: **manifest committed** (provenance), **weights gitignored**.
Eval baseline / leaderboard JSON	`eval_baselines/`	Tracked (survives a clean checkout); the no-regression gate reads it.
Committed regression fixture (video-free golden)	`eval_baselines/fixtures/<slug>/`	Tracked, **LF-pinned** (`.gitattributes`). Synthetic Tier-0 (or owner-cleared, de-attributed real). Drives the `golden_fixtures` characterization gate with **no video/GPU/DB**. See `docs/GOLDEN_REGRESSION_FIXTURES.md`; mint via `python -m backend.eval.golden_fixtures --freeze-synthetic` / `--freeze-real` (refusal-gated).
Pipeline run outputs (reels, trajectories, DBs, frames)	`output/`	**Gitignored.** Per-video isolation is the tracked **`PER_VIDEO_WORKSPACE.md`** project (P0 helper shipped #264; supersedes the archived `PER_VIDEO_OUTPUT_LAYOUT`).

Canonical files that stay at the repo ROOT (referenced by CI / packaging / docs ? do not move):
`pyproject.toml` (PEP 621 packaging, hatchling; `indexer` entry point) · `run_all_tests.py` (CI: `ci.yml`) ·
`run_regression.py` (the no-regression gate CLI, cited in `EXPERIMENT_HARNESS.md`) · `run_eval.py`
(grandfathered ? imported by `tests/test_eval_harness.py`). The old diagnostic `setup.py` moved

to
`scripts/doctor.py` (it shadowed the build system); invoke via `python scripts/doctor.py` or
`indexer --setup`.
New standalone run-drivers go in **`scripts/`**, not the root.

---

1. PIPELINE

1A. Runtime: video file -> highlight reel

```
typed config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-
defaults, never fatal)
                                        @backend/config/loader.py::load_config ->
@backend/config/models.py::AppConfig
  -> video_id = MD5(whole file)           @backend/utils/hashing.py::compute_video_id  (single
shared helper, 64KB chunks; imported by main + all run_*.py ? no more per-entrypoint copies)
  -> validate
@backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)
       FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity ->
SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see
@backend/pipeline/validators/__init__.py)
       [validation FAIL -> persist status="failed", return early; report STILL emitted in
finally]
  -> db.add_video(video_id, filepath, status, [r.dict()])   @backend/infrastructure/database.py
  -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
  -> seg = segmenter_registry.get(seg_name)(db, global_config)   ?
@backend/pipeline/segmenters/base.py::segmenter_registry  (400 + available list if missing)
  -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
       [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without
stitching]
  -> finally: emit_indexer_report(...)    @backend/reporting/__init__.py  (? ALWAYS runs ? even
on validation failure / exception; NEVER raises; grafts response["reports"])
  -> select segment ids -> [(start,end)] (+ stitching padding)
  -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name)
@backend/pipeline/stitcher/compiler.py
       per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy ->
output/<name>.mp4
```

Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):

```
P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
 -> P2 candidate "action windows" from motion/trajectory  [STOP: candidates persisted via
db.add_candidate_segment]
 -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip,
prompt)   ? provider_registry
 -> P4 db.add_play_segment + db.link_candidate_to_segment
```

Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) ->
add_play_segment.
Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground truth).

1B. WASB shuttle substrate (P2 of wasb_hybrid)

```
WasbHybridSegmenter.process_video               @backend/pipeline/segmenters/wasb_hybrid.py
 -> WasbRunner.healthcheck()  (gate; not ok -> return [])
@backend/pipeline/detectors/wasb_runner.py
 -> WasbRunner.run_predict(video, out_dir)
      stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python
wasb_infer.py ...'`
      -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU detector
pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
 -> TrackNetRunner.parse_trajectory_csv(csv)      @backend/pipeline/detectors/tracknet_runner.py
(shared, re-exported by WasbRunner)
```

```
 -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start,
velocity_thresh, merge_gap, min_window_duration, chunk_index)  [the model-agnostic WINDOWING
BRAIN]
```
(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width` fallback
(30.0, 1280.0).)

1C. Calibration / eval flow (NO pipeline run unless told)
```
GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]   stage ?
{candidates, final}
  -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
  -> harness.evaluate -> EvalReport -> report_to_dict
       -> batch.aggregate (corpus micro/macro)            @backend/eval/batch.py
       -> regression.regress(_with_provider) vs baseline   @backend/eval/regression.py  [CI
gate]
Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report, cross_validate}
@backend/eval/calibration.py
Distill Gemini->local: calibrate_local (thresholds) OR distill_local (learned classifier)
@backend/eval/{calibrate_local,distill_local}.py
Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
Gemini refinement driver (tuning, standalone): gemini_refine.{refine_window,full_video_rallies}
@backend/eval/gemini_refine.py
Audio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run
@backend/eval/audio/e1_probe.py
```
Entrypoints: `@run_eval.py` (single video score), `@scripts/run_phase3_eval.py` (manifest batch,
can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@scripts/nightly_regression.py` (nightly CACHED-trajectory CPU rally-quality tripwire, exit
1=regression/4=stale), `@scripts/nightly_reelcount.py` (nightly FULL-pipeline reel-count +
quality + cost on a GCP GPU VM, emailed), `@backend/eval/calibrate_wasb.py` +
`@backend/eval/export_wasb_segments.py` (WASB tuning drivers).

---

2. SUBSYSTEMS

2.0 Orchestration & config
- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI
mounts `/static` from `backend/api/static`. Logging is initialized with `basicConfig` (INFO,
timestamped format); backend modules use `logger = logging.getLogger(__name__)` (replaces stray
prints in hot paths like motion, stitcher, wasb_infer, config loader). User-facing CLI summaries
may still use `print` for direct output.
  - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module globals
set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every
endpoint NPEs.
  - **ASYNC (#16, PR-1):** `::process_video` `@POST /api/process` -> fast-fail 4xx
(path/exists/segmenter) -> `db.create_job` -> `_get_executor().submit(_drain_due_jobs)` -> **202
+ job_id**. `::_drain_due_jobs` / `::_run_claimed_job` (worker loop EXTRACTED to
`@backend/api/job_worker.py` (#234), re-exported on `backend.main`): claim each runnable job via
`db.claim_due_job` -> `_execute_processing` -> mark done/failed (a `JobWaiting` parks the job
via `set_job_waiting` for resume; EXECUTION_POLICY P3 self-scheduling). `::_execute_processing`
= the former sync body (hash -> validate -> add_video -> segment -> `finally:` always
`emit_indexer_report`, grafts `response["reports"]`; never raises from the report); a RAISING
segmenter now prune_after-restores pre-run rows before re-raising. `::get_job` `@GET
/api/jobs/{job_id}` -> `{status, progress, video_id, error, result, reports, ...}` ? job
`status` = EXECUTION state (`queued|running|done|failed`); the pipeline verdict lives in
`result["status"]`. `::_get_executor` lazy module-global `ThreadPoolExecutor(max_workers=1)`
(single box + Gemini serial-upload learning; tests monkeypatch it inline). `main()` runs
`db.fail_stale_jobs()` at startup (orphaned queued/running -> failed). `::compile_highlights`
`@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET
/api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs

`indexing.motion_threshold` (dual model/dict read). ? CLI `--video-path` mode still runs the pipeline synchronously inline (separate code path, unchanged by #16).
  - **UPLOAD/DOWNLOAD (PR-2, B2+B3):** the chunked-upload endpoints now live in `@backend/api/uploads.py` (#228 ? an `APIRouter` registered via `app.include_router`; `uploads.py::_upload_cfg` resolves the live `backend.main.global_config` lazily so it reads the startup config, and a lazy `import backend.main` avoids an import cycle) ? `uploads.py::upload_init` `@POST /api/upload/init` (`{filename, total_size_bytes?}` -> ext allowlist + size gate -> `{upload_id}`), `::upload_part` `@PUT /api/upload/{id}/part/{n}` (raw-bytes body, STRICTLY sequential n, streams to `.partial-<id>` under `security.upload_dir`; per-part/total cap breach -> 413 + upload ABORTED), `::upload_complete` `@POST .../complete` (`{total_parts}` -> assemble -> never-overwrite rename -> `{path, video_id(MD5), size_bytes}` ? client then POSTs /api/process with that path), `::upload_abort` `@DELETE /api/upload/{id}`. ? upload state (`uploads.py::_uploads` + `::_uploads_lock`) is IN-PROCESS ? restart aborts partials (client re-inits); per-user partitioning lands with PR-3. `main.py::download_highlight` `@GET /api/highlights/{video_id}/{filename}` (download was NOT moved ? still in main.py) ? safe-name + `resolve_under_root(output_dir)` then FileResponse stream (the old /api/compile alert path was browser-unreachable). Config: `SecurityConfig.{upload_dir='output/uploads', max_upload_mb=4096, max_part_mb=95, allowed_upload_extensions=[mp4,mov]}` (95MB: free-tier edge proxies cap bodies ~100MB).
  - **STUDIO / STORAGE endpoints (P1/P2/P3/P4/P6/P8, 2026-07-01):** `::capabilities` `@GET /api/capabilities` ? honest box probe; now includes a **`storage` block** (`storage_capabilities` in `@backend/storage/capabilities.py`: local/s3/gcs/gdrive `{id,kind,label,usable,free_gb}`, secret-free) alongside segmenters/ffmpeg/gpu/deployment. `::list_videos` `@GET /api/videos` (the "my videos" list, P6). `::get_source` `@GET /api/source/{id}` ? Range-capable FileResponse serving a **frame-identical annotation proxy** (D4), source fallback, background-warmed. `::post_annotations` `@POST /api/annotations` ? writes the verbatim annotator CSV to `output/labels/<video_id>.rallies.csv` (P8). `::create_asset` `@POST /api/assets` (**P4**, `{uploaded_path?|source_uri?, medium_uri, sport?}`) ? stores a video into a user-chosen medium (MD5-keyed `storage.put` for an upload?remote medium; register-in-place for local; at-rest URI = no copy), registers it (video_id=MD5) so it shows in `/api/videos`; **self-host posture only** (403 when `security.allowed_video_root` set); edge-auth mirrors `/api/process`. Cost ledger: `::get_job_cost` `@GET /api/jobs/{id}/cost`, `::get_video_cost` `@GET /api/videos/{id}/cost`. **Free-space guard (P2):** `@backend/utils/freespace.py::require_free_bytes` -> HTTP **507** in `uploads.py::upload_init` + the `/api/process` remote fetch-to-scratch; the WSL frame-extraction guard now **BLOCKS** (`detectors/base.py::_wsl_frame_stage_shortfall`, ~15× source).
  - `::load_config` ? ? legacy thin wrapper returning a **dict** (`load_app_config(path).model_dump(...)`); kept for transition ? new code imports `load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`/`::AnnotationsRequest`/`::AssetRequest` pydantic bodies.
  - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on `OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`. `compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is honored end-to-end (the old write-only `_runtime_overrides` hack is gone).
  - ? `--segmenter` argparse `choices` frozen at build time from `segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI default_segmenter fallback `'motion'`.
- `@scripts/run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()` (blocks automation), skips validation. Config via `backend.config.load_config` (typed); segmenter, padding, and db path all read from the model (no literal fallbacks).

10. user (2026-07-02T04:46:17.769Z)

```
1       """Cache-hygiene + retention sweep for the GPU highlight pipeline.
2
3       Scans the regenerable intermediates the pipeline leaves behind ? in the WSL stage
4       dir (``~/clips``), the Windows ``output/`` tree, and the upload landing zone
5       (``security.upload_dir``, default ``output/uploads``) ? classifies each entry as
6       **purge** (regenerable: frame caches, staged video copies, handoff clips, temp dirs;
7       and, past a retention window, uploaded source videos) or **keep** (durable: trajectory
8       CSVs, the SQLite DB, final reels) and reports the reclaimable space.
9
10      **Retention sweep (T11 ? pre-alpha-exposure):** uploaded source videos in the upload
11      dir are a *privacy*-driven purge, not a *disk*-driven one ? once a tester's footage is
```

```
    older than ``--uploads-retention-days`` (default 7) it is swept, along with the orphaned
    ``.partial-<id>`` temp files aborted uploads leave behind. Reels / DB / CSVs stay
durable
    per the cache-hygiene policy (the owner's call, 2026-06-21).

    DRY-RUN BY DEFAULT ? it only *shows* what it would delete. Pass ``--apply`` to delete.

        python -m tools.cleanup_caches                      # report only (safe)
        python -m tools.cleanup_caches --apply              # delete purge candidates
        python -m tools.cleanup_caches --keep-video 89605e80ed01   # protect one video's
cache
        python -m tools.cleanup_caches --older-than 7       # only purge regenerable caches
>7d old
        python -m tools.cleanup_caches --uploads-retention-days 3   # sweep uploads >3d old
        python -m tools.cleanup_caches --include-wasbcache  # also drop detector resume
caches
        python -m tools.cleanup_caches --json               # machine-readable plan (for a
cron/job wrapper)

    Pairs with the pipeline's own self-cleaning (indexing.{wasb,tracknet}.keep_frames=false,
    which deletes frames after a SUCCESSFUL run); this tool reclaims what failed/old runs
    and pre-self-cleaning videos left behind, and enforces the upload retention window
before
    non-owner (alpha) exposure. Stdlib only; safe to run anywhere (the WSL scan is skipped
    automatically when ``wsl`` is unavailable). ``--uploads-dir`` must match
``security.upload_dir``
    (its default matches the config default, ``output/uploads``).
    """

    from __future__ import annotations

    import argparse
    import os
    import shutil
    import subprocess
    import sys
    from dataclasses import dataclass
    from typing import Dict, List, Optional, Tuple

    # ---- classification (pure; unit-tested) ----------------------------------- #

    # (category, regenerable, default_purge). default_purge=True ? deleted unless a flag
    # protects it. wasbcache is regenerable but default_purge=False (it enables resume and
    # is tiny) ? opt in with --include-wasbcache.
    _KEEP = ("keep", False, False)


    def classify(name: str, is_dir: bool, location: str) -> Tuple[str, bool, bool]:
        """Classify a cache entry by name. Returns (category, regenerable,
default_purge)."""
        low = name.lower()
        if location == "uploads":
            # The upload landing zone (security.upload_dir). Uploaded source videos and the
            # orphaned `.partial-<id>` temp files aborted uploads leave behind are both
swept
            # past the retention window (default_purge=True; held by --uploads-retention-
days
            # in plan_deletions). Anything else here ? e.g. a future per-user subdir (PR-3)
? is
            # left untouched, since we don't recurse into unknown upload trees.
            if is_dir:
                return ("upload subdir", False, False)
            if low.startswith(".partial-"):
                return ("partial upload", True, True)  # aborted/in-flight chunk file
            if low.endswith((".mp4", ".avi", ".mov", ".mkv")):
```

```
                return ("uploaded source", True, True)  # tester footage ? privacy retention
            return ("other", False, False)
        if is_dir:
            if low.endswith("_frames"):
                return ("frame cache", True, True)
            if low.endswith("_wasbcache"):
                return (
                    "detector resume cache",
                    True,
                    False,
                )  # opt-in via --include-wasbcache
            if low.startswith("chunks_") and low.endswith("_handoff"):
                return ("ai handoff clips", True, True)
            if low.startswith("tmp"):
                return ("temp dir", True, True)
            return ("other dir", False, False)
        # files
        if low.endswith((".csv",)):
            return ("trajectory csv", False, False)  # durable analysis output
        if low.endswith((".db", ".sqlite", ".sqlite3")):
            return ("database", False, False)  # cached rallies ? never purge
        if low.endswith((".log", ".sh", ".txt", ".json", ".md")):
            return ("log/script", False, False)
        if low.endswith((".mp4", ".avi", ".mov", ".mkv")):
            # In the WSL stage dir, every video is a regenerable staged copy.
            if location == "wsl":
                return ("staged video", True, True)
            # In output/, a 1080p/proxy is regenerable; anything else is a produced reel.
            if "_1080p" in low or "_proxy" in low or low.endswith("_proxy.mp4"):
                return ("proxy video", True, True)
            return ("highlight reel", False, False)  # durable output ? keep
        return ("other", False, False)


    @dataclass
    class Entry:
        name: str
        path: str  # display path (WSL ~/clips/<name> or windows abs)
        size: int  # bytes
        mtime: float  # epoch seconds (0 if unknown)
        is_dir: bool
        location: str  # "wsl" | "windows" | "uploads"

        @property
        def category(self) -> Tuple[str, bool, bool]:
            return classify(self.name, self.is_dir, self.location)


    def plan_deletions(
        entries: List[Entry],
        *,
        keep_video: Optional[str],
        older_than_days: float,
        include_wasbcache: bool,
        now: float,
        uploads_retention_days: float = 0.0,
    ) -> Tuple[List[Entry], List[Entry]]:
        """Split entries into (to_delete, kept) per the flags. Pure ? no I/O.

        Two independent retention windows protect *recent* entries from purge:
          * ``uploads_retention_days`` guards the upload landing zone
(``location=="uploads"``)
            ? the privacy-driven T11 sweep window (default 7d at the CLI).
          * ``older_than_days`` guards the regenerable caches (everything else; default 0 =
off).
```

```
        A window of 0 means "no age guard" for that class. Entries with an unknown mtime (0)
        are never age-protected (treated as old).
        """
        to_delete: List[Entry] = []
        kept: List[Entry] = []
        for e in entries:
            cat, _regen, default_purge = e.category
            purge = default_purge or (include_wasbcache and cat == "detector resume cache")
            if purge and keep_video and keep_video in e.name:
                purge = False
            window = uploads_retention_days if e.location == "uploads" else older_than_days
            if purge and window > 0 and e.mtime > 0 and (now - e.mtime) < window * 86400.0:
                purge = False
            (to_delete if purge else kept).append(e)
        return to_delete, kept


    # ---- scanning (I/O) ------------------------------------------------------- #


    def human(n: int) -> str:
        f = float(n)
        for unit in ("B", "K", "M", "G", "T"):
            if f < 1024 or unit == "T":
                return f"{f:.1f}{unit}" if unit != "B" else f"{int(f)}B"
            f /= 1024
        return f"{f:.1f}T"


    def _wsl_lines(distro: str, cmd: str) -> List[str]:
        """Run a bash command in WSL and return stdout lines ([] on any failure)."""
        try:
            res = subprocess.run(
                ["wsl", "-d", distro, "bash", "-lc", cmd],
                capture_output=True,
                text=True,
                timeout=300,
            )
        except (OSError, subprocess.TimeoutExpired):
            return []
        return (res.stdout or "").splitlines()


    def scan_wsl(distro: str, stage_dir: str) -> List[Entry]:
        """List ~/clips entries with recursive size + mtime + type. [] if WSL absent.

        Uses ``du -sb`` (recursive byte sizes, native-fast on ext4) merged with ``find
        -printf`` (type + mtime) by basename. Deliberately avoids a ``for`` loop ? the
        user's login shell mangles loop variables, but du/find arg-globbing is reliable.
        """
        if not shutil.which("wsl"):
            return []
        size_lines = _wsl_lines(
            distro, f"du -sb {stage_dir}/* {stage_dir}/.[!.]* 2>/dev/null || true"
        )
        meta_lines = _wsl_lines(
            distro,
            f"find {stage_dir}/ -maxdepth 1 -mindepth 1 "
            f"-printf '%y\\t%T@\\t%f\\n' 2>/dev/null || true",
        )
        meta = {}  # basename -> (is_dir, mtime)
        for line in meta_lines:
            parts = line.split("\t")
            if len(parts) == 3:
                y, mt, name = parts
```

```
                try:
                    meta[name] = (y == "d", float(mt))
                except ValueError:
                    continue
        entries: List[Entry] = []
        for line in size_lines:
            if "\t" not in line:
                continue
            sz, path = line.split("\t", 1)
            name = path.rsplit("/", 1)[-1]
            try:
                size = int(sz)
            except ValueError:
                continue
            is_dir, mtime = meta.get(name, (False, 0.0))
            entries.append(
                Entry(
                    name=name,
                    path=f"{stage_dir}/{name}",
                    size=size,
                    mtime=mtime,
                    is_dir=is_dir,
                    location="wsl",
                )
            )
        return entries


    def _dir_size(path: str) -> int:
        total = 0
        for root, _dirs, files in os.walk(path):
            for fn in files:
                try:
                    total += os.path.getsize(os.path.join(root, fn))
                except OSError:
                    pass
        return total


    def scan_windows(output_dir: str, skip_dirs: Tuple[str, ...] = ()) -> List[Entry]:
        """List output/ entries (top level) plus the output/wasb sub-tree.

        ``skip_dirs`` paths are excluded at the top level so a dir scanned separately
        (e.g. the upload landing zone when it lives under output/) isn't double-counted
        here as an opaque "other dir" ? and so its retention is governed by scan_uploads,
        not silently kept whole.
        """
        skip = {os.path.realpath(p) for p in skip_dirs}
        entries: List[Entry] = []
        for base in (output_dir, os.path.join(output_dir, "wasb")):
            if not os.path.isdir(base):
                continue
            for de in os.scandir(base):
                # 'wasb' is scanned as its own base below ? don't also list it as a dir
here.
                if base == output_dir and de.name == "wasb":
                    continue
                if os.path.realpath(de.path) in skip:
                    continue
                try:
                    is_dir = de.is_dir()
                    size = _dir_size(de.path) if is_dir else de.stat().st_size
                    mtime = de.stat().st_mtime
                except OSError:
                    continue
```

```
                entries.append(
                    Entry(
                        name=de.name,
                        path=de.path,
                        size=size,
                        mtime=mtime,
                        is_dir=is_dir,
                        location="windows",
                    )
                )
        return entries


    def scan_uploads(uploads_dir: str) -> List[Entry]:
        """List the upload landing zone's top-level entries (location=="uploads").

        Flat by design today (uploads.py writes ``<upload_dir>/<filename>`` plus
        ``.partial-<id>`` temp files); a future per-user subdir (PR-3) shows up as an
        "upload subdir" and is left untouched until that retention is designed.
        """
        entries: List[Entry] = []
        if not os.path.isdir(uploads_dir):
            return entries
        for de in os.scandir(uploads_dir):
            try:
                is_dir = de.is_dir()
                size = _dir_size(de.path) if is_dir else de.stat().st_size
                mtime = de.stat().st_mtime
            except OSError:
                continue
            entries.append(
                Entry(
                    name=de.name,
                    path=de.path,
                    size=size,
                    mtime=mtime,
                    is_dir=is_dir,
                    location="uploads",
                )
            )
        return entries


    def _delete(e: Entry, distro: str) -> bool:
        try:
            if e.location == "wsl":
                subprocess.run(
                    ["wsl", "-d", distro, "bash", "-lc", f"rm -rf {_q(e.path)}"],
                    capture_output=True,
                    text=True,
                    timeout=600,
                    check=True,
                )
            elif e.is_dir:
                shutil.rmtree(e.path, ignore_errors=True)
            else:
                os.remove(e.path)
            return True
        except (OSError, subprocess.SubprocessError):
            return False


    def _q(path: str) -> str:
        if path.startswith("~/"):
            return "~/'" + path[2:].replace("'", "'\\''") + "'"
```

```
        return "'" + path.replace("'", "'\\''") + "'"


    def _report(title: str, rows: List[Entry]) -> int:
        print(f"\n{title}")
        if not rows:
            print("  (none)")
            return 0
        total = 0
        for e in sorted(rows, key=lambda x: x.size, reverse=True):
            cat = e.category[0]
            print(f"  {human(e.size):>8}  {cat:<22} {e.path}")
            total += e.size
        print(f"  {'-' * 8}")
        print(f"  {human(total):>8}  TOTAL ({len(rows)} items)")
        return total


    def _plan_summary(rows: List[Entry], now: float) -> List[dict]:
        """Structured per-entry rows (category/size/age) for --json / loud logging."""
        out = []
        for e in sorted(rows, key=lambda x: x.size, reverse=True):
            age_days = round((now - e.mtime) / 86400.0, 2) if e.mtime > 0 else None
            out.append(
                {
                    "path": e.path,
                    "category": e.category[0],
                    "location": e.location,
                    "size_bytes": e.size,
                    "age_days": age_days,
                }
            )
        return out


    def _by_category(rows: List[Entry]) -> List[Tuple[str, int, int]]:
        """(category, count, total_bytes) subtotals, largest-first ? the structured
breakdown."""
        agg: Dict[str, Tuple[int, int]] = {}
        for e in rows:
            cat = e.category[0]
            cnt, sz = agg.get(cat, (0, 0))
            agg[cat] = (cnt + 1, sz + e.size)
        return sorted(
            ((c, n, s) for c, (n, s) in agg.items()), key=lambda x: x[2], reverse=True
        )


    def main(argv: Optional[List[str]] = None) -> int:
        ap = argparse.ArgumentParser(
            description="Report/clean regenerable pipeline caches + sweep stale uploads
(dry-run by default)."
        )
        ap.add_argument(
            "--apply", action="store_true", help="actually delete (default: report only)"
        )
        ap.add_argument(
            "--keep-video",
            default=None,
            help="protect entries whose name contains this id/substring",
        )
        ap.add_argument(
            "--older-than",
            type=float,
            default=0.0,
```

```
386             help="only purge regenerable caches older than N days",
387         )
388         ap.add_argument(
389             "--include-wasbcache",
390             action="store_true",
391             help="also purge detector resume caches",
392         )
393         ap.add_argument(
394             "--stage-dir", default="~/clips", help="WSL stage dir (default ~/clips)"
395         )
396         ap.add_argument("--distro", default="Ubuntu", help="WSL distro (default Ubuntu)")
397         ap.add_argument(
398             "--output-dir", default="output", help="Windows output dir (default output)"
399         )
400         ap.add_argument(
401             "--uploads-dir",
402             default=None,
403             help="upload landing zone to sweep (default <output-dir>/uploads; match
security.upload_dir)",
404         )
405         ap.add_argument(
406             "--uploads-retention-days",
407             type=float,
408             default=7.0,
409             help="sweep uploaded source videos older than N days (default 7; 0=off). The T11
privacy window.",
410         )
411         ap.add_argument(
412             "--summary",
413             action="store_true",
414             help="print a single-line nudge (only if reclaimable >= --threshold-gb); for
hooks",
415         )
416         ap.add_argument(
417             "--threshold-gb",
418             type=float,
419             default=5.0,
420             help="--summary stays silent below this many reclaimable GB (default 5)",
421         )
422         ap.add_argument(
423             "--json",
424             action="store_true",
425             help="emit the machine-readable purge plan as JSON (for a cron/job wrapper) and
exit",
426         )
427         args = ap.parse_args(argv)
428 
429         import time
430 
431         now = time.time()
432         uploads_dir = args.uploads_dir or os.path.join(args.output_dir, "uploads")
433         entries = (
434             scan_wsl(args.distro, args.stage_dir)
435             + scan_windows(args.output_dir, skip_dirs=(uploads_dir,))
436             + scan_uploads(uploads_dir)
437         )
438         to_delete, kept = plan_deletions(
439             entries,
440             keep_video=args.keep_video,
441             older_than_days=args.older_than,
442             include_wasbcache=args.include_wasbcache,
443             now=now,
444             uploads_retention_days=args.uploads_retention_days,
445         )
446         reclaim = sum(e.size for e in to_delete)
```

```
447
448         if args.json:
449             import json
450
451             print(
452                 json.dumps(
453                     {
454                         "dry_run": not args.apply,
455                         "windows": {
456                             "uploads_retention_days": args.uploads_retention_days,
457                             "cache_older_than_days": args.older_than,
458                         },
459                         "uploads_dir": uploads_dir,
460                         "reclaimable_bytes": reclaim,
461                         "purge": _plan_summary(to_delete, now),
462                         "by_category": [
463                             {"category": c, "count": n, "bytes": s}
464                             for c, n, s in _by_category(to_delete)
465                         ],
466                     },
467                     indent=2,
468                 )
469             )
470             # --json is a plan view; deletion still requires a non-json --apply run (loud +
auditable).
471             return 0
472
473         if args.summary:
474             if reclaim >= args.threshold_gb * (1024**3):
475                 print(
476                     f"??  {human(reclaim)} reclaimable (regenerable cache + uploads >"
477                     f"{args.uploads_retention_days:g}d) ? run `python -m
tools.cleanup_caches` to review, `--apply` to delete."
478                 )
479             return 0
480
481         # Loud header: state the active retention windows up front so nobody mistakes a
482         # privacy sweep for a disk-cache cleanup (reels/DB/CSVs are NEVER purged here).
483         print("Retention windows in effect:")
484         print(
485             f"  uploads ({uploads_dir}): purge source videos >
{args.uploads_retention_days:g}d old"
486             + ("" if args.uploads_retention_days > 0 else "  [OFF]")
487         )
488         print(
489             "  caches  (frames/staged/proxies): "
490             + (
491                 f"purge > {args.older_than:g}d old"
492                 if args.older_than > 0
493                 else "purge all regenerable (no age guard)"
494             )
495         )
496         print(
497             "  KEEP (durable, never purged): highlight reels, the rally DB, trajectory CSVs,
logs/scripts"
498         )
499
500         reclaim = _report("PURGE candidates:", to_delete)
501         for cat, n, sz in _by_category(to_delete):
502             print(f"    · {human(sz):>8}  {cat} ({n})")
503         _report("KEEP (durable / protected):", kept)
504         print(f"\nReclaimable: {human(reclaim)}")
505
506         if not args.apply:
507             print(
```

```
508                 "\nDRY RUN ? nothing deleted. Re-run with --apply to delete the PURGE list."
509             )
510             return 0
511         if not to_delete:
512             print("\nNothing to delete.")
513             return 0
514 
515         print(f"\nDeleting {len(to_delete)} items...")
516         freed, failed = 0, 0
517         for e in to_delete:
518             if _delete(e, args.distro):
519                 freed += e.size
520                 print(f"  PURGED  {human(e.size):>8}  {e.category[0]:<18} {e.path}")
521             else:
522                 failed += 1
523                 print(f"  FAILED  {'':>8}  {e.category[0]:<18} {e.path}")
524         print(
525             f"Freed ~{human(freed)} across {len(to_delete) - failed} items"
526             + (f" ({failed} failed)" if failed else "")
527         )
528         return 0
529 
530 
531     if __name__ == "__main__":
532         sys.exit(main())
533 
```

11. user (2026-07-02T04:46:29.119Z)

```
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/config:
__init__.py
__pycache__
loader.py
models.py
presets.py
startup.py

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy/digitalocean:
README.md
bootstrap.sh
gpu_setup.sh
mount_scratch.sh

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/deploy/gcp:
README.md
buckets.env
create_gpu_vm.sh
nightly_regression
ops-agent-startup.sh
provision.sh
teardown.sh
Annotation Setup
badminton-highlight-indexer
khelsutra
khelsutra-screencast
rally-annotator
sports-data-collector
sports-obsreport
```

12. user (2026-07-02T04:46:30.344Z)

```
backend\flywheel.py:50:    adults-only attestation + a retention class ? lands with
**M9-consent**; until then this denies
backend\main.py:1808:def _maybe_sweep_gemini_orphans(config) -> None:
backend\main.py:1809:    """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a
```

```
previous crashed run.
backend\main.py:1811:    a failed sweep is logged and swallowed. The Gemini client is built with
an HTTP timeout so
backend\main.py:1826:                "[STARTUP] Gemini orphan sweep reclaimed %d remote
file(s).", deleted
backend\main.py:1829:        logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal):
%s", e)
backend\main.py:1999:    # P0e: refuse a SECOND instance against the same DB BEFORE the
destructive stale-job sweep below.
backend\main.py:2000:    # The job worker is in-process, so a concurrent process opening this DB
would sweep THIS one's
backend\main.py:2036:    # O1/P11: sweep Gemini File-API uploads orphaned by a hard process
death (the per-call
backend\main.py:2039:    _maybe_sweep_gemini_orphans(global_config)
backend\infrastructure\database.py:1137:            logger.error(f"Failed to sweep stale jobs:
{e}")
backend\providers\gemini.py:26:# sweep can distinguish OUR files from other apps sharing the
same Gemini API key
backend\providers\gemini.py:32:# hung TCP connection can never stall startup (the sweep) or
block a worker indefinitely.
backend\providers\gemini.py:60:        """Sweep and delete video files orphaned on the Gemini
File API by a previous
backend\providers\gemini.py:66:        considered, so the sweep never touches another app's
files under a shared API key
backend\providers\gemini.py:83:               "[Gemini GC] failed to list remote files
(skipping sweep): %s", e
backend\providers\gemini.py:104:               logger.debug("[Gemini GC] skipping a file during
sweep: %s", e)
backend\eval\eval_partial_labels.py:188:def sweep(
backend\eval\eval_partial_labels.py:239:        f"=== Config sweep vs human labels ({len(gts)}
rallies in "
backend\eval\eval_partial_labels.py:291:        "--sweep",
backend\eval\eval_partial_labels.py:295:    ap.add_argument("--top", type=int, default=12,
help="how many sweep rows to print")
backend\eval\eval_partial_labels.py:297:    if args.sweep:
backend\eval\eval_partial_labels.py:298:        sweep(
backend\utils\single_instance.py:3:The serving model runs an IN-PROCESS job worker, and
``fail_stale_jobs()`` sweeps any job left
backend\utils\single_instance.py:5:same SQLite DB (a second ``indexer --server``, or a CLI run
while a server is up), its sweep would
backend\eval\metrics.py:173:    """Score across a sweep of IoU thresholds (default 0.1..0.9 step
0.1).
backend\eval\per_user_eval.py:6:into a falsifiable criterion by sweeping the corpus size ``k``
and asking, at each ``k``,
backend\eval\per_user_eval.py:89:    """Sweep corpus size ``k`` and report where the candidate
first honestly beats control.
backend\eval\per_user_eval.py:107:    `ExperimentError` on unlabeled videos or a too-small
corpus (no valid ``k`` to sweep).
backend\eval\per_user_eval.py:119:        # n must exceed min_k. Too-few videos ? no sweepable
k; report an empty, honest curve.
backend\eval\segmentation_metrics.py:8:sweep). Reporting these alongside temporal-IoU is what
lets a reconciler's split/merge/
backend\eval\segmentation_metrics.py:41:#: Default tIoU sweep for mean average precision ? the
action-detection convention.
backend\eval\segmentation_metrics.py:118:    """Mean of ``average_precision`` over a tIoU sweep
(the standard mAP@tIoU)."""
backend\ui\assets\index-ukuS9wxa.js:60:[Omitted long matching line]
backend\eval\tolerance_metrics.py:6:boundary-error distributions + an over-seg guard
(split/merge counts) + a ?-sweep ? never one
backend\eval\tolerance_metrics.py:28:#: report the sweep alongside, since the golden END
convention carries >1.5 s of noise.
backend\eval\tolerance_metrics.py:31:SWEEP_TAUS: Tuple[float, ...] = (1.5, 2.0, 2.5, 3.0)
backend\eval\tolerance_metrics.py:336:def tolerance_sweep(
backend\eval\tolerance_metrics.py:337:    preds: List[Interval], gts: List[Interval], taus:
Tuple[float, ...] = SWEEP_TAUS
```

```
backend\eval\tolerance_metrics.py:339:    """Symmetric-? F1 sweep ? the trend curve (a single ?
misleads). Monotonic non-decreasing in ?."""
```

13. user (2026-07-02T04:46:36.982Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       Unknown / typo'd config keys are rejected at load time via extra="forbid".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, List, Literal, Optional
12
13      from pydantic import BaseModel, Field, field_validator, model_validator
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          pts_max_gap_seconds: float = Field(
31              default=10.0,
32              ge=0.0,
33              description=(
34                  "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35                  "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36                  "a gap >10s suggests edited/missing content. Mobile phones, screen "
37                  "recordings, and other encoders may use different GOP sizes ? tune "
38                  "this threshold for your recording device."
39              ),
40          )
41          relevance: ValidationRelevanceConfig = Field(
42              default_factory=ValidationRelevanceConfig
43          )
44
45
46      class TrackNetConfig(BaseModel):
47          wsl_distro: str = "Ubuntu"
48          repo_dir: str = "~/models/TrackNetV4"
49          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50          conda_env: str = "TrackNetV4"
51          weights_path: str = ""
52          queue_length: int = 5
53          stage_in_wsl: bool = True
54          wsl_stage_dir: str = "~/clips"
55          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56          velocity_threshold: float = 0.02
57          # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58          # decoded frame cache) are deleted automatically ? they are pure intermediates,
59          # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60          # true only for debugging. On FAILURE they are always kept so a re-run can resume.
```

```
61          keep_frames: bool = False
62          # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
63          # ~15×-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
64          # 0 = no extra floor (the headroom guard still applies).
65          min_free_gb: float = 0.0
66
67
68      class WasbIndexConfig(BaseModel):
69          """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71          Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72          these knobs actually take effect ? previously this whole block was silently dropped
73          because nested config sections defaulted to extra='ignore'.
74          """
75
76          wsl_distro: str = "Ubuntu"
77          repo_dir: str = "~/models/WASB-SBDT"
78          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79          conda_env: str = "wasb"
80          weights_path: str = (
81              "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82          )
83          sport: str = "badminton"
84          wsl_stage_dir: str = "~/clips"
85          timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86          velocity_threshold: float = 0.02
87          # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
---
88          # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89          # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90          # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91          device: str = "cuda"  # auto | cuda | mps | cpu
92          python_bin: str = (
93              "python"  # the `wasb` conda env python (invoked by path, no `conda activate`)
94          )
95          # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96          # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97          # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98          keep_frames: bool = False
99          # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100         # ~15×-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101         # 0 = no extra floor (the headroom guard still applies).
102         min_free_gb: float = 0.0
103         # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104         # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105         # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106         # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107         # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108         # Tri-state:
109         #   None       = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110         #                behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
```

```
        #   True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
        stream_video: Optional[bool] = None


    class FusionConfig(BaseModel):
        """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).

        Every default reproduces control behaviour: the fusion segmenter persists the
        shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
is
        true ? the person-cue features, but it does NOT gate the served handoff unless a
        threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
mask
        is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
only);
        supplied ? off-court persons (spectators / adjacent courts) are excluded.
        """

        # Which trajectory substrate seeds the windows (shuttle stays primary).
        substrate: Literal["wasb", "tracknet"] = "wasb"
        # Windowing velocity threshold for the fusion family (the engine reads
        # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
        # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
        velocity_threshold: float = 0.02
        # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
        # enabled ? so the default path never imports torch / touches the GPU.
        cue_flags: dict[str, bool] = Field(default_factory=dict)
        # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
        # 0 = OFF (no gating; persisted candidates are always the full superset for offline
re-scoring).
        min_crossings: int = Field(default=0, ge=0)
        # Served Gate B: when the person cue is on, drop windows with median in-court
players
        # < this. 0 = OFF.
        min_players: int = Field(default=0, ge=0)
        # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
        court_polygon: Optional[list[list[float]]] = None
        # Net line for the person two-sided-motion split (person features only ? the shuttle
        # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
        net_axis: Literal["x", "y"] = "y"
        net_position: float = Field(default=0.5, ge=0.0, le=1.0)


    class IndexingConfig(BaseModel):
        default_segmenter: str = "yolo_hybrid"
        use_gpu: bool = True
        # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
        # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
        # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
        # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
        # RALLY_DETECTOR_IMPL env var overrides this. See
pipeline/detectors/{stub,native_wasb}_runner.py.
        detector_impl: str = "auto"
        motion_threshold: float = 0.08
        min_segment_duration: float = 3.0
        dead_time_merge_gap: float = 2.0
        chunk_duration: float = 90.0
        chunk_overlap: float = 10.0
        detector_model: str = "fasterrcnn_resnet50_fpn"
        detector_score_threshold: float = 0.5
        yolo_velocity_threshold: float = 0.15
        yolo_frame_skip: int = 3
```

```
167         yolo_tracker: str = "bytetrack.yaml"
168         yolo_prefetch_workers: int = 2
169         min_action_window_duration: float = 2.0
170         # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
171         # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
172         # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
173         # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
174         # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
175         # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
176         # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
177         max_window_duration: float = 6.0
178         window_min_split_gap: float = (
179             0.5  # min internal gap (s) that qualifies as a split point
180         )
181         # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
182         # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
183         # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
184         # Only cuts (recall can't drop). OFF by default until measured/promoted.
185         window_hard_cap: bool = False
186         # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
187         # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
188         # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
189         # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
190         # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
191         # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
192         # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
193         window_inactivity_end: float = 1.5
194         inactivity_rally_thresh: float = 0.20
195         inactivity_min_density: float = 0.30
196         # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
197         # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
198         # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
199         # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
200         # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
201         # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
202         # regression), |?end| 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),
203         # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
204         # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
205         inactivity_temporal_density: bool = True
206         # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
```

```
blob: after
207         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
208         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
209         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
210         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
211         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
212         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
213         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
214         window_blob_fallback: bool = False
215         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
216         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
217         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
218         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
219         window_blob_factor: float = 4.0
220         log_yolo_candidates: bool = True
221         store_yolo_candidates: bool = True
222         ai_handoff_padding: float = 2.0
223         ai_max_workers: int = 5
224         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
225         # chunks of a high-bitrate (4K) source blow past the provider upload cap
226         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
227         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
228         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
229         # Matches gemini_refine's --clip-scale-width default.
230         ai_chunk_scale_width: int = 1280
231         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
232         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
233         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
234         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
235         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
236         skip_ai_handoff: bool = False
237         # Optional debug knob: trim every video to the first N seconds before processing.
238         debug_duration: Optional[float] = None
239
240         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
241         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
242         fusion: FusionConfig = Field(default_factory=FusionConfig)
243
244         @field_validator("default_segmenter")
245         @classmethod
246         def segmenter_must_be_registered(cls, v: str) -> str:
247             # Lazy import: the segmenter registry imports modules that may import
248             # config, so we resolve it at validation time rather than at module load.
249             from backend.pipeline.segmenters.base import segmenter_registry
250
251             if segmenter_registry.get(v) is None:
252                 available = sorted(segmenter_registry.list_available().keys())
253                 raise ValueError(
254                     f"Segmenter '{v}' is not registered. Available: {', '.join(available)}"
255                 )
256             return v
```

```
257
258         @model_validator(mode="after")
259         def _chunk_step_must_be_positive(self) -> "IndexingConfig":
260             # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
261             # step makes `while t < duration: t += step` loop forever, growing memory before
any
262             # GPU work. Reject the bad config at load time instead.
263             if self.chunk_overlap >= self.chunk_duration:
264                 raise ValueError(
265                     f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
266                     f"({self.chunk_duration}); otherwise chunking never advances."
267                 )
268             return self
269
270
271     class OutputConfig(BaseModel):
272         default_highlights_name: str = "highlights.mp4"
273         db_name: str = "sports_indexer.db"
274         # Directory where the DB, highlight reels, and processed clips are written.
275         # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
276         output_dir: str = "output"
277
278
279     class WatermarkConfig(BaseModel):
280         """Branding overlay burned into each highlight clip during the per-clip re-encode
281         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
282         (under `stitching.watermark` in config.json) to turn it off. The text is fully
283         configurable. The concat stage stays stream-copy (-c copy)."""
284
285         enabled: bool = True
286         text: str = "Generated by Khelsutra.guru"
287         logo_path: str = ""  # optional transparent PNG overlay; "" = no logo
288         font_path: str = ""  # optional .ttf; "" = use the fontconfig `font` family below
289         font: str = "Sans"  # fontconfig family used when font_path is empty
290         font_size_ratio: float = Field(
291             default=0.035, gt=0.0, le=0.5
292         )  # text height as a fraction of frame height
293         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
294         box: bool = True  # faint backing box behind the text for legibility
295         margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
296         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
297             "bottom-right"
298         )
299
300
301     class StitchingConfig(BaseModel):
302         begin_padding: float = 0.5
303         end_padding: float = 0.5
304         use_cache: bool = False
305         min_shot_count: int = 1
306         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
307         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
308         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
309         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
310         # local detector over-fires and its false positives are mostly SHORT (motion blips,
311         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
312         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
313         min_rally_duration: float = Field(default=0.0, ge=0.0)
```

```
        watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


    class LoggingConfig(BaseModel):
        """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""

        # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
        # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
        # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
        # them to WARNING; set true to restore full chatter for request-flow debugging.
        verbose_http: bool = False


    class SecurityConfig(BaseModel):
        # When set, /api/process video_path must resolve under this directory (hosted/tenant
        # mode). Default None preserves the single-user local 'any local file' workflow.
        allowed_video_root: Optional[str] = None

        # --- Chunked upload (B2, PR-2) ---
        # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
        # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
        # contain upload_dir or /api/process will reject the uploaded paths.
        upload_dir: str = "output/uploads"
        # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
        # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
        max_upload_mb: int = 4096
        max_part_mb: int = 95
        allowed_upload_extensions: List[str] = ["mp4", "mov"]

        # --- M9 edge auth (Cloudflare Access, D9) ---
        # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
        # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
        # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
        # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
        # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
        # application's audience tag. NO secrets here ? these are public identifiers.
        edge_auth_enabled: bool = False
        cf_team_domain: str = (
            ""  # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
        )
        cf_access_aud: str = ""  # the CF Access application AUD tag the token must carry
        # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
        # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
        # OFF ? Cf-Access is a header, not a cookie.)


    class StorageConfig(BaseModel):
        """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
        behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
        + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
        Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
        **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
        (boto3/google auth chains supply credentials). See ``backend/storage/``."""

```

```
368         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
369         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
370         output_root: str = ""
371         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
372         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
373         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
374         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
375         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
376         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
377         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
378         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
379         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
380         input_root: str = ""
381         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
382         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
383         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
384         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
385         # the legacy flat layout is used unchanged.
386         single_folder_per_video: bool = False
387         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
388         workspace_root: str = ""
389         # Cloud namespace prefix under the root so per-video folders sit tidily beside
390         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
391         workspace_prefix: str = "workspaces"
392         local: dict[str, Any] = Field(default_factory=dict)
393         s3: dict[str, Any] = Field(default_factory=dict)
394         gcs: dict[str, Any] = Field(default_factory=dict)
395         gdrive: dict[str, Any] = Field(default_factory=dict)
396         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
397         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
398         gdrive_api: dict[str, Any] = Field(default_factory=dict)
399
400
401     class DeploymentConfig(BaseModel):
402         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
403         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
404
405         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
406         behaves exactly as before this section existed. A profile is opt-in (``config.json``
407         ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
408         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
409         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
410         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
411
412         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
```

```
413
414         # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
415         profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
416         # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
417         # (recorded for observability; consumed by the ComputeBackend seam from M2+).
418         compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
419
420
421     class BillingConfig(BaseModel):
422         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
423
424         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
425         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
426         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
427         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
428         inserts a real, calibrated version and points ``pricebook_version`` at it.
429
430         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
431         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
432         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
433
434         enabled: bool = False
435         pricebook_version: str = (
436             "placeholder-v0"  # the seeded $0 version; owner flips to a real one
437         )
438         fx_inr_per_usd: float = 83.0  # USD?INR display rate (Bangalore market)
439         margin: float = 0.0  # resale markup (?0); 0 = bill at cost
440
441
442     class FlywheelConfig(BaseModel):
443         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
444         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
445         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so
446         the owned model climbs toward the D11 ship floor.
447
448         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
449         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
450         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
451         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-raw).
452         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
453         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
454         owner/counsel/GPU scope. Pairs with M9-consent."""
455
456         enabled: bool = False
457         workspace_prefix: str = (
458             "flywheel"  # per-video workspace prefix for captured artifacts
459         )
460         shadow_eval: bool = True  # run the owned-vs-Gemini shadow comparison on capture
```

```
461         min_promote_n: int = 5  # promote_if_served_safe `min_n` (the served-safe gate)
462
463
464     class AppConfig(BaseModel):
465         """Root configuration object.
466
467         All runtime configuration should be accessed via an instance of this class
468         (or the loader) rather than raw dict lookups.
469         """
470
471         sport: Sport = "badminton"
472         ai_provider: str = "gemini"
473         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
474         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
475         ai_model: str = "gemini-flash-latest"
476
477         validation: ValidationConfig = Field(default_factory=ValidationConfig)
478         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
479         output: OutputConfig = Field(default_factory=OutputConfig)
480         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
481         security: SecurityConfig = Field(default_factory=SecurityConfig)
482         logging: LoggingConfig = Field(default_factory=LoggingConfig)
483         storage: StorageConfig = Field(default_factory=StorageConfig)
484         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
485         billing: BillingConfig = Field(default_factory=BillingConfig)
486         flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)
487
488         # Reject unknown keys ? typed config migration is complete (D3).
489         model_config = {
490             "extra": "forbid",
491             "validate_assignment": True,
492         }
493
494
495     # Convenience type alias for callers that want to annotate
496     Config = AppConfig
497
```

14. user (2026-07-02T04:46:49.896Z)

```
1780                 print(
1781                     "  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play."
1782                 )
1783             else:
1784                 print(
1785                     "  Result: [DEAD TIME] Idle movement. Segment classified as dead time."
1786                 )
1787         else:
1788             print("  [ERROR] No visual frames were extracted in the sample range.")
1789
1790         print("=" * 60)
1791
1792
1793     def _enforce_startup_or_exit(config) -> None:
1794         """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
1795         any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
1796         the exposed server (``include_exposure=True``, M9 hardening), a half-configured
exposure (edge
1797         auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no
```

```
        ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
        the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
        fail-fast wiring + the default-OFF no-op are unit-testable."""
        try:
            enforce_startup_checks(config, logger=logger, include_exposure=True)
        except StartupCheckError as e:
            logger.error("[STARTUP] refusing to start: %s", e)
            sys.exit(1)


    def _maybe_sweep_gemini_orphans(config) -> None:
        """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous
crashed run.
        A no-op unless the configured provider is Gemini AND its API key is present. Never
raises ?
        a failed sweep is logged and swallowed. The Gemini client is built with an HTTP
timeout so
        a hung connection can't stall startup indefinitely. Extracted for unit tests."""
        try:
            provider_name = getattr(config, "ai_provider", "")
            if provider_name != "gemini":
                return
            api_key = os.environ.get("GEMINI_API_KEY")
            if not api_key:
                return
            provider = provider_registry.get(
                "gemini", api_key=api_key, model_name=config.ai_model
            )
            deleted = provider.cleanup_orphaned_files()
            if deleted:
                logger.info(
                    "[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted
                )
        except Exception as e:
            logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)


    def main():
        parser = argparse.ArgumentParser(
            description="Sports Highlight Indexer Orchestrator"
        )
        parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
        parser.add_argument(
            "--output-dir",
            default="output",
            help="Directory where processed clips/highlights are saved",
        )
        parser.add_argument(
            "--setup",
            action="store_true",
            help="Launch dependency checker and setup utility",
        )
        parser.add_argument(
            "--debug-clip",
            type=float,
            help="Timestamp (in seconds) of a video clip to run detailed diagnostics on",
        )
        parser.add_argument(
            "--server", action="store_true", help="Run the FastAPI local API server"
        )
        parser.add_argument(
            "--app",
            action="store_true",
```

```
            help="Launch the app: start the local server (localhost-only) and open the "
            "bundled UI in your browser. Falls back to a free port if --port is busy.",
        )
        parser.add_argument(
            "--port", type=int, default=8000, help="Port to run the FastAPI server on"
        )
        parser.add_argument(
            "--host",
            default=None,
            help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0 when "
            "security.edge_auth_enabled. Expose on a network ONLY behind an auth gate.",
        )
        parser.add_argument(
            "--max-frames",
            type=int,
            help="Limit processing to the first N sub-sampled frames for debugging",
        )

        available_segmenters = list(segmenter_registry.list_available().keys())
        parser.add_argument(
            "--segmenter",
            choices=available_segmenters,
            help=f"Choose rally detection segmenter. Available: {available_segmenters}",
        )
        parser.add_argument(
            "--trim-start",
            type=float,
            help="Start time in seconds for the debug trim segment feature",
        )
        parser.add_argument(
            "--trim-end",
            type=float,
            help="End time in seconds for the debug trim segment feature",
        )
        parser.add_argument(
            "--trim-output",
            help="Output filename for the debug trimmed segment (saves in output-dir unless
absolute path)",
        )

        args = parser.parse_args()

        if args.setup:
            # Invoke the diagnostics/bootstrap script (renamed from the old root
            # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
            # build system). Resolve its path from this file so `indexer --setup`
            # works regardless of the caller's cwd.
            import subprocess

            doctor_path = os.path.join(
                os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
                "scripts",
                "doctor.py",
            )
            print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
            subprocess.run([sys.executable, doctor_path])
            return

        if args.trim_start is not None and args.trim_end is not None:
            if not args.video_path:
                print("[ERROR] Please provide --video-path to extract a segment.")
                return

            # Determine output path
            out_filename = args.trim_output or "trimmed_segment.mp4"
```

```
1922            if os.path.isabs(out_filename):
1923                out_path = out_filename
1924                out_dir = os.path.dirname(out_path)
1925                out_name = os.path.basename(out_path)
1926            else:
1927                out_dir = resolve_output_dir(
1928                    args.output_dir
1929                )  # P0a: app-home aware (CWD-identical when unset)
```

15. user (2026-07-02T04:46:55.572Z)

```
1929                )  # P0a: app-home aware (CWD-identical when unset)
1930                out_path = os.path.join(out_dir, out_filename)
1931                out_name = out_filename
1932
1933            os.makedirs(out_dir, exist_ok=True)
1934            print(
1935                f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}..."
1936            )
1937
1938            # global_config isn't initialized this early; load the typed config so the trim
1939            # clip carries the same configured stitching paddings + watermark as a real
1940            # highlight (watermark default-on).
1941            stitching = load_app_config().stitching
1942            compiler = VideoCompiler(
1943                out_dir,
1944                begin_padding=stitching.begin_padding,
1945                end_padding=stitching.end_padding,
1946                watermark=stitching.watermark,
1947            )
1948            try:
1949                res_path = compiler.compile_highlights(
1950                    args.video_path, [(args.trim_start, args.trim_end)], out_name
1951                )
1952                print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
1953            except Exception as e:
1954                print(f"[ERROR] Failed to compile trimmed segment: {e}")
1955            return
1956
1957        if args.debug_clip is not None:
1958            if not args.video_path:
1959                print("[ERROR] Please provide --video-path to debug a specific clip.")
1960                return
1961            run_cli_diagnose_clip(args.video_path, args.debug_clip)
1962            return
1963
1964        # P2b (PACKAGING_PLAN.md gate O8): batteries-included first run for the friendly
`--app`
1965        # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
1966        # no API key, no GPU, no cloud bills) at the resolved app-home
(default_config_path(), which
1967        # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
1968        # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER
overwrites an
1969        # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
1970        # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
1971        if args.app:
1972            seeded_path, seeded = write_light_default_config()
1973            if seeded:
1974                print(
```

```
                    f"[APP] First run: seeded a truly-local default config at {seeded_path}
"
                    f"(detector 'motion' ? offline, no API key or GPU needed). "
                    f"Use the in-app setup to switch to Gemini."
                )

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig
        _enforce_startup_or_exit(
            global_config
        )  # M5: fail fast on an incoherent ACTIVE profile

        # The CLI --output-dir overrides the configured output directory. It now lives
        # on the typed model (OutputConfig.output_dir), so every consumer ? including
        # /api/compile ? reads the same value instead of a write-only runtime hack.
        # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
        # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
        global_config.output.output_dir = resolve_output_dir(args.output_dir)
        os.makedirs(global_config.output.output_dir, exist_ok=True)

        db_path = os.path.join(
            global_config.output.output_dir, global_config.output.db_name
        )

        # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
        # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
        # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
        # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
        global _instance_lock
        try:
            _instance_lock = acquire_lock(db_path + ".lock")
        except OSError as exc:
            # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
            # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
            # legitimate single instance).
            logger.warning(
                "[JOBS] Could not create the single-instance lock at %s.lock (%s);
proceeding "
                "WITHOUT the second-instance guard.",
                db_path,
                exc,
            )
            _instance_lock = None
        else:
            if _instance_lock is None:
                print(
                    f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
                    f"start ? a second instance would corrupt in-flight job state. Stop the
other one, or "
                    f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance."
                )
                sys.exit(1)

        db_instance = Database(db_path)

```

```
2028        # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
2029        # the executor is in-process. Mark them failed so pollers see a terminal state.
2030        swept = db_instance.fail_stale_jobs()
2031        if swept:
2032            logger.warning(
2033                f"[JOBS] Swept {swept} stale job(s) from a previous run ? failed."
2034            )
2035
2036        # O1/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
2037        # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
2038        # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
2039        _maybe_sweep_gemini_orphans(global_config)
2040
2041        # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
2042        if args.video_path and not args.server and not args.app:
2043            print(f"[CLI] Processing video file: {args.video_path}")
2044            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
2045            # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
2046            # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
2047            storage_cfg = getattr(global_config, "storage", None)
2048            try:
2049                input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
2050            except ValueError as e:
2051                print(f"[FAIL] {e}")
2052                return
2053            if input_ref.backend == "local" and not os.path.exists(input_ref.key):
2054                print(f"[FAIL] Video file not found: {args.video_path}")
2055                return
2056            local_video = storage.acquire_local_input(args.video_path, storage_cfg)
2057
2058            video_id = compute_video_id(local_video)
2059            was_reingest = db_instance.get_video(video_id) is not None
2060
2061            # Validate (with-context variant surfaces ffprobe_meta for the report)
2062            print("[CLI] Validating...")
2063            val_results, val_context = registry.run_all_with_context(
2064                local_video, global_config
2065            )
2066            failures = [r for r in val_results if not r.passed]
2067
2068            # Save video status
2069            status = "failed" if failures else "processed"
2070            db_instance.add_video(
2071                video_id, args.video_path, status, [r.model_dump() for r in val_results]
2072            )
2073
2074            seg_name = args.segmenter or getattr(
2075                getattr(global_config, "indexing", None), "default_segmenter", "motion"
2076            )
2077            segmenter_actual = None
2078            segmentation_ran = False
2079            try:
2080                print("\n" + "=" * 40)
2081                print("VALIDATION SUMMARY")
2082                print("=" * 40)
2083                for r in val_results:
2084                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
2085                    print(f"{status_symbol} {r.validator_name}: {r.message}")
```

```
2086                print("=" * 40 + "\n")
2087
2088                if failures:
2089                    print("[FAIL] Video failed checks. Indexing halted.")
2090                    return
2091
2092                # Index
2093                segmenter_cls = segmenter_registry.get(seg_name)
2094                if not segmenter_cls:
2095                    print(f"[FAIL] Segmenter '{seg_name}' not found.")
2096                    return
2097
2098                print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
2099                segmenter_actual = seg_name
2100                play_wm, cand_wm = db_instance.segment_watermarks(video_id)
2101                segmenter = segmenter_cls(db_instance, global_config)
2102                result = segmenter.process_video(
2103                    video_id, local_video, max_frames=args.max_frames
2104                )
2105                segmentation_ran = True
2106
2107                if isinstance(result, Failure):
2108                    print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
2109                    print("[FAIL] Indexing halted for video.")
2110                    segments = []
2111                else:
2112                    segments = result.value if hasattr(result, "value") else result
2113                    print(
2114                        f"[SUCCESS] Process complete. Indexed {len(segments)} play segments
inside database: {db_path}"
2115                    )
2116                # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
2117                _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
2118            finally:
2119                # Always emit the per-video observability report. Never raises.
2120                paths = emit_indexer_report(
2121                    db=db_instance,
2122                    video_id=video_id,
2123                    # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI would
write a
2124                    # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source URI
is kept
2125                    # in DB provenance via db.add_video.
2126                    video_path=local_video,
2127                    config=global_config,
2128                    val_results=val_results,
```

16. user (2026-07-02T04:47:01.751Z)

```
2129                    val_context=val_context,
2130                    segmenter_requested=args.segmenter,
2131                    segmenter_actual=segmenter_actual,
2132                    was_reingest=was_reingest,
2133                    segmentation_ran=segmentation_ran,
2134                )
2135                if paths:
2136                    print(f"[REPORT] Technical report : {paths.get('technical_md')}")
2137                    print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
2138                    print(f"[REPORT] Machine JSON     : {paths.get('json')}")
2139            return
2140
2141        # Start Local Server Mode
2142        if args.app or args.server or not args.video_path:
2143            if args.app:
```

```
                # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
                # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
                bind_host = "127.0.0.1"
                port = _resolve_app_port(bind_host, args.port)
                if port != args.port:
                    print(f"[APP] Port {args.port} is busy ? using {port} instead.")
                url = f"http://{bind_host}:{port}/"
                # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
                # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
                # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
                print(
                    f"[APP] Starting KhelSutra ? opening {url} in your browser "
                    f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop."
                )
                threading.Thread(
                    target=_open_browser_when_ready,
                    args=(url, url + "api/health"),
                    daemon=True,
                ).start()
            else:
                bind_host = _resolve_bind_host(
                    global_config, args.host
                )  # P0c: config/profile-driven (default 127.0.0.1)
                port = args.port
                print(
                    f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}..."
                )
                if bind_host not in ("127.0.0.1", "localhost", "::1"):
                    print(
                        f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
                        f"gate (edge_auth_enabled / a reverse proxy) is in front, and set "
                        f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header guard
defaults to "
                        f"loopback-only and will 400 forwarded requests otherwise)."
                    )
            # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
            # in-process wake-timer for any parked 'waiting' job died with the old process,
so
            # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
            _get_executor().submit(_drain_due_jobs, global_config, db_instance)
            # Stamp shared state on the app so route handlers access it via
request.app.state
            # instead of the Optional module-level globals (CODE_CLEANUP_AUDIT P5 / D2).
            create_app(global_config, db_instance)
            uvicorn.run(app, host=bind_host, port=port)


    if __name__ == "__main__":
        # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
        # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
        # module object. If `main()` runs here it configures `__main__`'s globals, leaving
        # `backend.main.db_instance` = None; the async-job drain then crashes silently on
        # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
        # `backend.main` so `main()` configures the module everyone actually reads. (No
```

```
recursion:
2195        # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
2196        import backend.main as _bm
2197
2198        _bm.main()
2199
```

17. user (2026-07-02T04:47:08.724Z)

```
1       # Cloud Run GPU serving ? build / deploy / run (COMPUTE_DECOUPLED_SERVING M6/M7)
2
3       The cloud-serving compute path: a control plane (`CloudRunCompute`,
`backend/compute/cloud_run.py`)
4       dispatches a Cloud Run **Job** execution running this image's worker on an **L4**, then
**bucket-polls**
5       a done-marker for completion (D16). Scale-to-zero (D7) ? no warm pool.
6
7       > **Status:** the dispatch shim + image are **landed + mock-tested** (M6). The **real
build + push +
8       > run on an L4** is the **M7** owner step, within the **$100/?10k** budget (D17) ?
confirm cmd + hourly
9       > cost first, **delete every resource after** ([[gcp-vm-always-delete]]). CI does
**not** build this image.
10
11      ## 0. Prerequisites (owner)
12      - A GCP project with billing on + the Cloud Run, Artifact Registry, and Cloud Build APIs
enabled.
13      - GPU quota for L4 (`GPUS_ALL_REGIONS` ? 1) in the chosen region (asia-south1, else
Singapore ? see
14        `deploy/gcp/README.md`; L4 is often stocked-out, fall back per that runbook).
15      - The WASB weights staged in the bucket (WASB-C3 unresolved ? owner-gated):
`gs://<bucket>/models/wasb_badminton_best.pth.tar`.
16
17      ## 1. Build + push the image (Artifact Registry)
18      ```bash
19      PROJECT=<gcp-project>; REPO=rally
20      # L4 GPU is region-limited ? use asia-southeast1 (Singapore), NOT asia-south1 (Mumbai).
21      REGION=asia-southeast1
22      gcloud artifacts repositories create $REPO --repository-format=docker --location=$REGION
2>/dev/null || true
23      # `gcloud builds submit` has NO -f flag, so a non-root Dockerfile needs a build config:
24      gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project $PROJECT .
25      ```
26      *(A CUDA + micromamba image is large; the first build is ~8?10 min. Budget-watch per
D17.)*
27
28      ## 2. Create the Cloud Run **Job** (GPU, scale-to-zero)
29      ```bash
30      gcloud run jobs create rally-worker \
31        --image $REGION-docker.pkg.dev/$PROJECT/$REPO/$IMG:latest \
32        --region $REGION --gpu 1 --gpu-type nvidia-l4 --cpu 4 --memory 16Gi \
33        --max-retries 0 --task-timeout 3600 \
34        --set-env-vars WASB_WEIGHTS=/staged/wasb_badminton_best.pth.tar
35      # (stage the weights into the container at start, or fetch from gs:// in an init step ?
WASB-C3.)
36      ```
37
38      ## 3. Wire the dispatcher (control plane)
39      The control plane runs with the **cloud-serving** profile (`DEPLOYMENT_PROFILE=cloud-
serving` ?
40      `compute_target=cloud_run`). `get_compute_backend(config)` then builds `CloudRunCompute`
from env:
41      ```bash
42      export DEPLOYMENT_PROFILE=cloud-serving
```

```
43      export CLOUD_RUN_JOB=rally-worker  CLOUD_RUN_REGION=$REGION
GOOGLE_CLOUD_PROJECT=$PROJECT
44      # A `worker infer` (or the API job path) with a gs:// video now DISPATCHES to the Job
instead of
45      # running in-process; the container runs the SAME worker INLINE (compute_target forced
local_gpu).
46      python -m backend.worker infer --video-uri gs://<bucket>/videos/clip.mp4 --out-uri
gs://<bucket> --segmenter wasb_hybrid
47      ```
48
49      ## 4. M7 acceptance (the owner runlog)
50      Capture a `DEPLOY_REPORT_cloud-serving_<date>.md` under
51      `docs/COMPUTE_DECOUPLED_SERVING/runlogs/` (mirrors the truly-local template): the exact
commands,
52      rally counts, `gpu_active_seconds`/`wall_seconds`/`bytes_out`, a reel smoke-test
(`ffmpeg -f null -`
53      exit 0 + a 5-frame contact sheet), gotchas, and the **$0 teardown audit**.
54
55      ## Gotchas
56      - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
57        + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
58      - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
59        ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
60      - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker**
carrying the
61        real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min
hang on a bad
62        video). Only a **hung/crashed** container (no marker at all) makes the bounded poll
time out
63        (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and
the API path
64        surfaces it as a `cloud dispatch error` job failure (not a traceback).
65      - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
66      - **`GOOGLE_CLOUD_PROJECT` is required** for the real client ?
`GoogleCloudRunJobClient.run_job`
67        fails fast with a clear `RuntimeError` if it is unset/empty (it must resolve a real
`job_path`;
68        an unset project would otherwise build a bogus `projects/None/...` name). Export it
(step 3) before
69        dispatching. The guard runs before the `google-cloud-run` import, so it is exercised
in CI even
70        though the SDK isn't a CI/test dependency.
71
```

18. user (2026-07-02T04:47:10.366Z)

```
__init__.py
__pycache__
test_ablation.py
test_annotation_providers.py
test_annotation_proxy.py
test_api_annotations.py
test_api_assets.py
test_api_cloud_dispatch.py
test_api_endpoints.py
test_app_home.py
test_app_launcher.py
test_async_jobs.py
test_audio_features.py
test_automated_provider.py
```

```
test_base.py
test_billing.py
test_bind_and_host.py
test_calibrate_local.py
test_calibrate_wasb.py
test_calibration.py
test_check_md_links.py
test_cleanup_caches.py
test_compiler.py
test_compute_backend.py
test_config.py
test_config_deployment.py
test_config_wasb.py
test_court_gate.py
test_database.py
test_db_upgrade_safety.py
test_detector_keying.py
test_device_context.py
test_distill_local.py
test_doctor.py
test_edge_auth.py
test_eval_annotations.py
test_eval_batch.py
test_eval_classifier.py
test_eval_harness.py
test_eval_metrics.py
test_eval_partial_labels.py
test_eval_regression.py
test_eval_stats.py
test_eval_training.py
test_execution_policy.py
test_experiment.py
test_experiment_honest.py
test_export_wasb_segments.py
test_ffmpeg_resolver.py
test_flywheel.py
test_freespace.py
test_fusion_compare.py
test_fusion_features.py
test_fusion_golden_telemetry.py
test_fusion_hybrid.py
test_gemini.py
test_gemini_refine.py
test_generate_highlights.py
test_geometry.py
test_golden_fixtures.py
---
221 C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-
indexer/tests/test_cleanup_caches.py
```

19. user (2026-07-02T04:47:15.138Z)

```
1       """Unit tests for the cache-hygiene tool's pure logic (classify + plan_deletions).
2
3       The I/O (wsl/du/find scan, deletion) is not unit-tested here ? these guard the
4       classification + filtering rules that decide what gets deleted, which is the part
5       where a mistake would be destructive.
6       """
7
8       from tools.cleanup_caches import Entry, classify, plan_deletions
9
10
11      def test_classify_regenerable_intermediates():
12          assert (
13              classify("GX010125_1080p__abc_frames", True, "wsl")[2] is True
```

```
        )  # frame cache -> purge
        assert (
            classify("GX010125_1080p__abc.mp4", False, "wsl")[2] is True
        )  # staged video -> purge
        assert classify("GX010125_1080p.mp4", False, "windows")[2] is True  # proxy -> purge
        assert (
            classify("chunks_gemini_handoff", True, "windows")[2] is True
        )  # handoff clips -> purge
        assert classify("tmp9xk2", True, "windows")[2] is True  # temp dir -> purge


    def test_classify_durable_kept():
        assert classify("adarsh_traj.csv", False, "wsl")[2] is False  # trajectory -> keep
        assert classify("sports_indexer.db", False, "windows")[2] is False  # DB -> keep
        assert (
            classify("GX010125_highlights.mp4", False, "windows")[2] is False
        )  # reel -> keep
        assert classify("run_wasb.sh", False, "wsl")[2] is False  # script -> keep
        # wasbcache is regenerable but opt-in only (default_purge False)
        cat, regen, default_purge = classify("adarsh_traj_wasbcache", True, "wsl")
        assert cat == "detector resume cache" and regen is True and default_purge is False


    def _e(name, size, is_dir, location="wsl", mtime=0.0):
        return Entry(
            name=name,
            path=f"~/clips/{name}",
            size=size,
            mtime=mtime,
            is_dir=is_dir,
            location=location,
        )


    def test_plan_default_purges_frames_keeps_csv_and_wasbcache():
        entries = [
            _e("v_frames", 90_000, True),
            _e("v.mp4", 2_000, False),
            _e("v_traj.csv", 100, False),
            _e("v_traj_wasbcache", 500, True),
        ]
        to_del, kept = plan_deletions(
            entries,
            keep_video=None,
            older_than_days=0,
            include_wasbcache=False,
            now=1_000_000.0,
        )
        names = {e.name for e in to_del}
        assert names == {"v_frames", "v.mp4"}
        assert {e.name for e in kept} == {"v_traj.csv", "v_traj_wasbcache"}


    def test_plan_include_wasbcache_opts_it_in():
        entries = [_e("v_traj_wasbcache", 500, True)]
        to_del, _ = plan_deletions(
            entries,
            keep_video=None,
            older_than_days=0,
            include_wasbcache=True,
            now=1_000_000.0,
        )
        assert [e.name for e in to_del] == ["v_traj_wasbcache"]


```

```
    def test_plan_keep_video_protects_matching_substring():
        entries = [_e("GX010125_frames", 90_000, True), _e("kushagra_frames", 5_000, True)]
        to_del, kept = plan_deletions(
            entries,
            keep_video="GX010125",
            older_than_days=0,
            include_wasbcache=False,
            now=1_000_000.0,
        )
        assert [e.name for e in to_del] == ["kushagra_frames"]
        assert [e.name for e in kept] == ["GX010125_frames"]


    def test_plan_older_than_protects_recent():
        now = 1_000_000.0
        entries = [
            _e("recent_frames", 90_000, True, mtime=now - 3600),  # 1h old
            _e("old_frames", 90_000, True, mtime=now - 10 * 86400),  # 10d old
        ]
        to_del, kept = plan_deletions(
            entries, keep_video=None, older_than_days=7, include_wasbcache=False, now=now
        )
        assert [e.name for e in to_del] == ["old_frames"]
        assert [e.name for e in kept] == ["recent_frames"]


    # ---- T11 retention sweep: the upload landing zone --------------------------- #


    def _u(name, size, is_dir=False, mtime=0.0):
        return Entry(
            name=name,
            path=f"output/uploads/{name}",
            size=size,
            mtime=mtime,
            is_dir=is_dir,
            location="uploads",
        )


    def test_classify_uploads_location():
        assert (
            classify("match1.mp4", False, "uploads")[2] is True
        )  # uploaded source -> purge
        assert classify("clip.MOV", False, "uploads")[2] is True  # case-insensitive ext
        assert (
            classify(".partial-deadbeef", False, "uploads")[2] is True
        )  # aborted partial -> purge
        assert (
            classify("user_42", True, "uploads")[2] is False
        )  # future per-user subdir -> keep
        assert (
            classify("notes.txt", False, "uploads")[2] is False
        )  # unknown file -> untouched
        assert classify("match1.mp4", False, "uploads")[0] == "uploaded source"
        assert classify(".partial-x", False, "uploads")[0] == "partial upload"


    def test_plan_uploads_retention_protects_recent_purges_old():
        now = 1_000_000.0
        entries = [
            _u(
                "fresh.mp4", 5_000, mtime=now - 2 * 86400
            ),  # 2d old  -> protected (window 7)
            _u("stale.mp4", 5_000, mtime=now - 9 * 86400),  # 9d old  -> swept
```

```
            _u(".partial-abc", 10, mtime=now - 9 * 86400),  # old orphan -> swept
            _u(".partial-live", 10, mtime=now - 60),  # in-flight  -> protected
        ]
        to_del, kept = plan_deletions(
            entries,
            keep_video=None,
            older_than_days=0,
            include_wasbcache=False,
            now=now,
            uploads_retention_days=7,
        )
        assert {e.name for e in to_del} == {"stale.mp4", ".partial-abc"}
        assert {e.name for e in kept} == {"fresh.mp4", ".partial-live"}


    def test_plan_uploads_window_independent_of_cache_older_than():
        # Uploads use their OWN window; the cache --older-than must not protect old uploads,
        # and the uploads window must not protect old regenerable caches.
        now = 1_000_000.0
        entries = [
            _u("upload.mp4", 5_000, mtime=now - 3 * 86400),  # 3d < uploads(7) -> kept
            _e(
                "old_frames", 9_000, True, location="windows", mtime=now - 3 * 86400
            ),  # 3d, caches no guard -> purged
        ]
        to_del, kept = plan_deletions(
            entries,
            keep_video=None,
            older_than_days=0,
            include_wasbcache=False,
            now=now,
            uploads_retention_days=7,
        )
        assert [e.name for e in to_del] == ["old_frames"]
        assert [e.name for e in kept] == ["upload.mp4"]


    def test_plan_uploads_retention_zero_means_off():
        now = 1_000_000.0
        entries = [_u("fresh.mp4", 5_000, mtime=now - 60)]  # 1 min old
        to_del, _ = plan_deletions(
            entries,
            keep_video=None,
            older_than_days=0,
            include_wasbcache=False,
            now=now,
            uploads_retention_days=0,
        )
        assert [e.name for e in to_del] == [
            "fresh.mp4"
        ]  # no window -> swept regardless of age


    def test_by_category_subtotals_largest_first():
        from tools.cleanup_caches import _by_category

        cats = _by_category([_u("a.mp4", 100), _u("b.mp4", 200), _u(".partial-x", 5)])
        assert cats[0] == ("uploaded source", 2, 300)  # largest bytes first
        assert ("partial upload", 1, 5) in cats


    def test_scan_skips_nested_uploads_dir(tmp_path):
        # The uploads dir lives under output/; scan_windows must NOT also list it as an
        # opaque "other dir" (that would double-count it and shield it from retention).
        from tools.cleanup_caches import scan_uploads, scan_windows
```

```
209
210         out = tmp_path / "output"
211         up = out / "uploads"
212         up.mkdir(parents=True)
213         (out / "GX_frames").mkdir()  # a regenerable cache dir in output/
214         (up / "match.mp4").write_bytes(b"x" * 10)  # an uploaded source video
215         (up / ".partial-z").write_bytes(b"y" * 3)  # an orphan partial
216
217         win = scan_windows(str(out), skip_dirs=(str(up),))
218         ups = scan_uploads(str(up))
219         assert {e.name for e in win} == {"GX_frames"}  # uploads dir not double-listed
220         assert {e.name for e in ups} == {"match.mp4", ".partial-z"}
221         assert all(e.location == "uploads" for e in ups)
222
```

20. user (2026-07-02T04:47:56.690Z)

```
1       """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3       Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4       proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
5       under security.upload_dir, then submitted to /api/process by the returned path.
6       Upload state is in-process by design (this is the single-box alpha): a server restart
7       aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8       arrives with PR-3 identity scoping.
9
10      The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11      ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12      reads the live value the server set at startup (and so tests that monkeypatch
13      ``backend.main.global_config`` flow through unchanged).
14      """
15
16      import os
17      import threading
18      import uuid
19      from typing import Any, Dict, Optional
20
21      from fastapi import APIRouter, HTTPException, Request
22      from pydantic import BaseModel
23
24      from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
25      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
26      from backend.utils.paths import safe_output_filename
27
28      router = APIRouter()
29
30      _uploads: Dict[str, Dict[str, Any]] = {}
31      _uploads_lock = threading.Lock()
32
33
34      class UploadInitRequest(BaseModel):
35          filename: str
36          total_size_bytes: Optional[int] = None
37
38
39      class UploadCompleteRequest(BaseModel):
40          total_parts: int
41
42
43      def _upload_cfg():
44          # Resolved lazily (import inside the function) to read the LIVE startup config that
45          # main() sets on the module global, and to avoid a circular import at module load
46          # (main.py imports this router; this module must not import main at import time).
47          import backend.main as _main
48
```

```
        sec = getattr(_main.global_config, "security", None)
        upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
        max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
        part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
        exts = [
            e.lower().lstrip(".")
            for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
        ]
        return upload_dir, max_bytes, part_bytes, exts


    def _abort_upload(upload_id: str) -> None:
        with _uploads_lock:
            st = _uploads.pop(upload_id, None)
        if st:
            try:
                os.remove(st["tmp_path"])
            except OSError:
                pass


    @router.post("/api/upload/init")
    def upload_init(req: UploadInitRequest):
        """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
        upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
        try:
            name = safe_output_filename(req.filename)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        ext = os.path.splitext(name)[1].lower().lstrip(".")
        if ext not in exts:
            raise HTTPException(
                status_code=400,
                detail=f"Extension '.{ext}' not allowed. Allowed: {sorted(exts)}",
            )
        if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
            raise HTTPException(
                status_code=413,
                detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
            )
        os.makedirs(upload_dir, exist_ok=True)
        # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
effort ? an
        # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
(never blocks a
        # legit upload); the per-part total cap in upload_part is the hard backstop either
way.
        require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
        upload_id = uuid.uuid4().hex
        tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
        open(tmp_path, "wb").close()
        with _uploads_lock:
            _uploads[upload_id] = {
                "filename": name,
                "tmp_path": tmp_path,
                "parts": 0,
                "bytes": 0,
            }
        return {
            "upload_id": upload_id,
            "max_part_mb": part_bytes // (1024 * 1024),
            "next_part": f"/api/upload/{upload_id}/part/0",
        }


```

```
    @router.put("/api/upload/{upload_id}/part/{index}")
    async def upload_part(upload_id: str, index: int, request: Request):
        """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
        _, max_bytes, part_bytes, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if index != st["parts"]:
            raise HTTPException(
                status_code=409,
                detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
sequential)",
            )

        received = 0
        overflow = None
        with open(st["tmp_path"], "ab") as fh:
            async for chunk in request.stream():
                received += len(chunk)
                if received > part_bytes:
                    overflow = (
                        f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
                    )
                    break
                if st["bytes"] + received > max_bytes:
                    overflow = (
                        f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
                    )
                    break
                fh.write(chunk)
        if overflow:
            _abort_upload(upload_id)
            raise HTTPException(
                status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
            )

        with _uploads_lock:
            st["parts"] += 1
            st["bytes"] += received
        return {
            "upload_id": upload_id,
            "received_parts": st["parts"],
            "received_bytes": st["bytes"],
        }


    @router.post("/api/upload/{upload_id}/complete")
    def upload_complete(upload_id: str, req: UploadCompleteRequest):
        """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
        upload_dir, _, _, _ = _upload_cfg()
        with _uploads_lock:
            st = _uploads.get(upload_id)
        if st is None:
            raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
        if st["parts"] == 0 or st["parts"] != req.total_parts:
            raise HTTPException(
                status_code=409,
                detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
            )
        final_path = os.path.join(upload_dir, st["filename"])
        if os.path.exists(
            final_path
        ):  # never overwrite an earlier upload with the same name
```

```
174             stem, ext = os.path.splitext(st["filename"])
175             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
176         os.replace(st["tmp_path"], final_path)
177         with _uploads_lock:
178             _uploads.pop(upload_id, None)
179         video_id = compute_video_id(final_path)
180         return {
181             "path": os.path.abspath(final_path),
182             "video_id": video_id,
183             "size_bytes": os.path.getsize(final_path),
184             "next": "POST /api/process with this path",
185         }
186 
187 
188     @router.delete("/api/upload/{upload_id}")
189     def upload_abort(upload_id: str):
190         """Abort an in-flight upload and delete its partial file."""
191         with _uploads_lock:
192             known = upload_id in _uploads
193         if not known:
194             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
195         _abort_upload(upload_id)
196         return {"status": "aborted", "upload_id": upload_id}
197 
```

21. user (2026-07-02T04:48:07.856Z)

```
docs\ANALYZERS_AND_RECONCILERS.md:379:(auditability + the repo's honesty/attribution working
agreement). The pipeline either:
docs\ANALYZERS_AND_RECONCILERS.md:486:**Honesty caveat (attribution working agreement).** No
external results or citations are asserted here. A
docs\data_pipeline\VIDEO_RECORDING_GUIDELINES.md:6:> **This is a tracked thought, not a finished
standard.** The exact numeric thresholds below are **placeholders** to be replaced with values
measured against our golden set. See "How we'll set the real numbers" (§3). Captured now (per
the team's working agreement) so it isn't lost; to be revisited.
docs\UI_ALPHA_EXECUTION_PLAN.md:58:## Working agreement
```

22. user (2026-07-02T04:48:09.743Z)

```
__init__.py
__pycache__
annotation_proxy.py
app_home.py
ffmpeg.py
freespace.py
geometry.py
hashing.py
logging_setup.py
paths.py
redact.py
run_telemetry.py
single_instance.py
video.py
workspace.py
---
```

Lint-only lane (no `ruff format` enforcement ? a whole-repo reformat would bury blame history for zero behavior value).

```
#
```

Selected: pyflakes + pycodestyle errors (ruff defaults) + pycodestyle warnings (W) + bugbear (B) + isort (I ? consistent import ordering).

```
target-version = "py38"
```

scratch/ = throwaway debugging scripts that may hit real APIs; it is also excluded from pytest collection (see pytest.ini).

```
extend-exclude = ["scratch"]

[lint]
extend-select = ["W", "B", "I"]

[lint.per-file-ignores]
```

Entrypoint scripts deliberately run setup (load_dotenv, logging config, sys.path) before importing backend modules ? import order is load-bearing.

```
"backend/main.py" = ["E402"]
"scripts/run_phase3_eval.py" = ["E402"]
"scripts/run_process_and_stitch.py" = ["E402"]
[mypy]
```

Lenient, GREEN baseline (quality sweep AI-4): the gate protects the clean modules and pins an explicit quarantine list below ? not a typing crusade. Ratchet: when you touch a quarantined module, try removing its entry.

```
ignore_missing_imports = True
```

backend/ is a namespace package (no root __init__.py) ? required so mypy maps files to backend.* module names instead of duplicating top-level names.

```
explicit_package_bases = True
```

--- Quarantine (known-loose modules, each with a reason) ---

Optional module globals (db_instance/global_config set by create_app() / main()). Route handlers use request.app.state exclusively; helpers take explicit params. backend.api.job_worker lifted from quarantine (now takes explicit config/db params).

```
[mypy-backend.main]
ignore_errors = True
```

Worker dispatcher: composes the SAME validator/segmenter/report building blocks a backend.main, which are typed `config: dict` but receive the AppConfig migration shir at runtime (the .get() compat layer). Shares backend.main's config-shim quarantine until the typed-config migration removes the dict?AppConfig duality everywhere.

```
[mypy-backend.worker.__main__]
ignore_errors = True
```

yolo_hybrid: the fusion-P1 extraction moved its loose torchvision/cv2 inline code to detectors/person_detector.py (which IS type-clean, no quarantine). What remains here is pre-existing orchestration debt unrelated to that move (implicit-Optional process_video defaults, the Success|Failure ABC-override mismatch, a Future[...] annotation) ? ratchet those out in a focused typing pass, not here. (wasb_hybrid/tracknet_hybrid left the quarantine with the AI-7 collapse.)

```
[mypy-backend.pipeline.segmenters.yolo_hybrid]
ignore_errors = True

[mypy-backend.pipeline.segmenters.gemini]
ignore_errors = True
```

Registry builds cls(None, {}) probes; typing it honestly needs a Protocol.

```
[mypy-backend.pipeline.segmenters.base]
ignore_errors = True
[pytest]
```

When `pytest` is invoked without an explicit path, only collect from tests/. scratch/ holds debugging scripts that hit real APIs (Gemini upload, etc.) and should never run as part of the regular test suite.

```
testpaths = tests
```

23. user (2026-07-02T04:48:22.773Z)

```
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
aa915a7 deploy/gcp: centralize project + bucket names in one sourced buckets.env (#410)
52d97b5 P4: POST /api/assets ? store a video into the user's chosen medium (#409)
7ea6ea9 P3: /api/capabilities reports usable storage backends (#408)
f28af73 P2: engine free-space guard (HTTP 507 + WSL frame-stage BLOCK) (#407)
759a67f feat(api): frame-preserving annotation proxy for /api/source (D4) (#406)
5a5c4fa feat(api): P8a ? GET /api/source + POST /api/annotations (annotate-in-UI backend) (#405)
4a7bfbe docs: record the launch NO-GO decision (owner, 2026-06-30) (#404)
6e973af docs: fix stale rally-cluster claims (no archival ? cluster still gated) (#403)
af1ccbe chore(#399): launch-PR process hardening ? link-check CI, PR template, doc cross-links
(#402)
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (fetch)
origin  https://github.com/Khelsutra/badminton-highlight-indexer.git (push)
github.com
  ? Logged in to github.com account avidullu (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_**********************************
```

24. user (2026-07-02T04:48:22.899Z)

```
Found 12 files
docs\CODE_MAP.md
docs\archives\past_projects\DECOUPLED_COMPUTE\06-RISKS-AND-OPEN-QUESTIONS.md
docs\archives\PER_VIDEO_OUTPUT_LAYOUT-SUPERSEDED-2026-06-21.md
docs\PLATFORM_ARCHITECTURE.md
docs\PER_VIDEO_WORKSPACE.md
docs\COMPUTE_DECOUPLED_SERVING\README.md
docs\BACKLOG.md
docs\UI_ALPHA_EXECUTION_PLAN.md
docs\VIDEO_LOCALITY_MODEL.md
docs\STORAGE_SHARING_MODEL.md
docs\POST_166_WORK_ITEMS_MILESTONES_AND_RISK_ASSESSMENT.md
docs\archives\MONETIZATION_AUDIT.md
```

25. assistant (2026-07-02T04:48:28.683Z)

Now let me set up the isolated worktree per the branching safety protocol.

26. user (2026-07-02T04:48:34.935Z)

```
Preparing worktree (new branch 'r2-8-retention-sweep-scheduler')
branch 'r2-8-retention-sweep-scheduler' set up to track 'origin/master'.
HEAD is now at e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe
details) (#413)
r2-8-retention-sweep-scheduler
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
```

27. assistant (2026-07-02T04:50:06.847Z)

Now I have the full picture. Let me check main.py's import section before writing code.

28. user (2026-07-02T04:50:11.125Z)

```
import argparse
import csv
import json
import logging
import math
import os
import shutil
import socket
import sys
import tempfile
import threading
import uuid
import webbrowser
from importlib.metadata import PackageNotFoundError
from importlib.metadata import version as _pkg_version
from urllib.parse import quote

import uvicorn
from dotenv import load_dotenv
```

PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME

launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None) ke

exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare load_doten

stdlib-only import ? keeps this pre-pydantic line light.

```
from backend.utils.app_home import env_file_to_load, resolve_output_dir

load_dotenv(env_file_to_load())
```

Centralized logging setup for the indexer (hot paths use logger, CLI output uses print

```
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
    datefmt="%H:%M:%S",
)
logger = logging.getLogger(__name__)
from typing import Any, Dict, List, Optional, Tuple

from fastapi import FastAPI, HTTPException, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
from starlette.middleware.trustedhost import TrustedHostMiddleware

from backend import (
    billing,  # M8: per-job cost ledger (USD internal / INR display)
    storage,  # M2: URI-aware input acquisition (local = identity passthrough)
)
from backend.api import (
    auth as edge_auth,  # M9: Cloudflare Access edge-auth (default-OFF)
)
from backend.config import (
    AppConfig,
    StartupCheckError,
    StitchingConfig,
```

```
    enforce_startup_checks,
    write_light_default_config,  # P2b: batteries-included first-run config seeding
)
from backend.config import (  # new typed config
    load_config as load_app_config,
)
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.compiler import VideoCompiler
from backend.pipeline.stitcher.watermark_i18n import localize_watermark
from backend.pipeline.validators.base import registry
from backend.providers import provider_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure  # standardized error/result contract
from backend.storage.capabilities import (
    storage_capabilities,  # P3: secret-free "which media can this box use" report
)
from backend.utils.ffmpeg import (  # P2a: single ffmpeg/ffprobe resolver
    ffmpeg_bin,
    ffprobe_bin,
)
from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
from backend.utils.paths import (
    output_base,
    resolve_under_root,
    safe_output_filename,
    validate_input_path,
)
from backend.utils.redact import (
    redact_secrets,  # P0b: mask secret-shaped fields in /api/config
)
from backend.utils.single_instance import acquire_lock  # P0e: one instance per database

app = FastAPI(title="Sports Highlight Indexer API")
```

29. assistant (2026-07-02T04:50:19.649Z)

Now I'll write the changes. First, the config model.

30. user (2026-07-02T04:50:28.831Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

31. user (2026-07-02T04:50:36.199Z)

```
355         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
356         # backend/main.py so importing the app does no filesystem I/O; default ["*"].
Credentials stay
357         # OFF ? Cf-Access is a header, not a cookie.)
358
359
360     class StorageConfig(BaseModel):
361         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
362         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
363         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
364         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
365         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
366         (boto3/google auth chains supply credentials). See ``backend/storage/``."""
```

```
367
368         backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
369         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
370         output_root: str = ""
371         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
372         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
373         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
374         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
375         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
376         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
377         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
378         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
379         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
380         input_root: str = ""
381         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
382         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>/]<video_id>/...
383         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
384         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
385         # the legacy flat layout is used unchanged.
386         single_folder_per_video: bool = False
387         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
388         workspace_root: str = ""
389         # Cloud namespace prefix under the root so per-video folders sit tidily beside
390         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
391         workspace_prefix: str = "workspaces"
392         local: dict[str, Any] = Field(default_factory=dict)
393         s3: dict[str, Any] = Field(default_factory=dict)
394         gcs: dict[str, Any] = Field(default_factory=dict)
395         gdrive: dict[str, Any] = Field(default_factory=dict)
396         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
397         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
398         gdrive_api: dict[str, Any] = Field(default_factory=dict)
399
400
401     class DeploymentConfig(BaseModel):
402         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
403         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
404
```

32. user (2026-07-02T04:50:48.794Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-
sweep/backend/config/models.py has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

33. user (2026-07-02T04:50:55.159Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
```

indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-sweep/backend/config/models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

34. assistant (2026-07-02T04:51:00.819Z)

Now the sweeper module.

35. assistant (2026-07-02T04:51:29.833Z)

```
"""In-server uploads-zone retention sweep (T11 / #301 / audit R2-8).

The standalone tool (``python -m tools.cleanup_caches``) knows how to classify and
purge stale uploads, but nothing schedules it ? so on a hosted box the upload landing
zone (``security.upload_dir``) grows without bound and the 7-day privacy retention
window (owner policy, 2026-06-21) is unenforced. This module gives the server an
OPT-IN periodic sweep of the **uploads zone only** (the WSL stage dir and the
``output/`` cache zones stay with the tool / operator):

* ``retention.sweep_enabled`` (default **False**) ? nothing runs unless the owner
  flips it; the default server boot is byte-identical to before.
* ``retention.sweep_interval_hours`` (default 24) ? cadence of the daemon-thread
  timer; a sweep also runs once at startup so a rarely-restarted box still catches up.
* ``retention.uploads_retention_days`` (default 7) ? the T11 privacy window. Uploaded
  source videos and orphaned ``.partial-*`` files older than this are deleted.
  ? ``<= 0`` makes the sweep a NO-OP ? deliberately SAFER than the CLI tool, where a
  0 window means "no age guard" (sweep everything now).

The sweep REUSES the tool's classify/plan/scan/purge helpers by import (no shelling
out), so the decision logic stays single-sourced and unit-tested in one place. Every
purge is logged loudly ? one ``[RETENTION]`` line per deleted entry plus a summary ?
so a hosted box's journal shows exactly what was deleted and why (the observability
working agreement). Retention is time-window-only: uploads referenced by non-terminal
jobs are NOT exempted (#301's open design call ? owner to ratify a job-aware guard).

The systemd-timer alternative (running the standalone tool on a schedule instead of
enabling the in-server sweep) lives in ``deploy/systemd/``.
"""

from __future__ import annotations

import logging
import threading
import time
from typing import Any, Callable, Dict, Optional

from tools.cleanup_caches import _delete, human, plan_deletions, scan_uploads

logger = logging.getLogger(__name__)


def run_uploads_sweep(
    uploads_dir: str, retention_days: float, *, now: Optional[float] = None
) -> Dict[str, Any]:
    """One T11 sweep pass over ``uploads_dir``: purge entries older than the window.

    Pure-decision core reused from tools.cleanup_caches (classify via Entry.category
    inside ``plan_deletions``); ``now`` is injectable for fake-clock tests. Returns a
    summary dict (also logged) and never raises on individual deletion failures.
    """
    now = time.time() if now is None else now
    if retention_days <= 0:
        # An unattended daemon must never read "window off" as "purge everything now"
        # (the CLI's 0 = no-age-guard semantics). Refuse loudly instead.
        logger.warning(
```

```
            "[RETENTION] uploads_retention_days=%g is not a positive window ? "
            "skipping the sweep (nothing deleted).",
            retention_days,
        )
        return {"skipped": True, "purged": 0, "failed": 0, "kept": 0, "freed_bytes": 0}

    entries = scan_uploads(uploads_dir)
    to_delete, kept = plan_deletions(
        entries,
        keep_video=None,
        older_than_days=0.0,  # scan_uploads emits only location=="uploads" entries,
        include_wasbcache=False,  # so only the uploads window below applies.
        now=now,
        uploads_retention_days=retention_days,
    )
    freed = 0
    failed = 0
    for e in to_delete:
        age_days = (now - e.mtime) / 86400.0 if e.mtime > 0 else float("inf")
        if _delete(e, distro=""):  # location is never "wsl" here ? distro unused
            freed += e.size
            logger.info(
                "[RETENTION] PURGED %s (%s, %s, %.1fd old > %gd window)",
                e.path,
                e.category[0],
                human(e.size),
                age_days,
                retention_days,
            )
        else:
            failed += 1
            logger.warning(
                "[RETENTION] FAILED to purge %s (%s) ? will retry next sweep",
                e.path,
                e.category[0],
            )
    summary: Dict[str, Any] = {
        "skipped": False,
        "purged": len(to_delete) - failed,
        "failed": failed,
        "kept": len(kept),
        "freed_bytes": freed,
    }
    logger.info(
        "[RETENTION] uploads sweep of %s: purged %d (%s freed), failed %d, kept %d "
        "(window %gd)",
        uploads_dir,
        summary["purged"],
        human(freed),
        failed,
        summary["kept"],
        retention_days,
    )
    return summary


def _sweep_loop(
    stop_event: threading.Event, interval_sec: float, sweep_once: Callable[[], Any]
) -> None:
    """Run ``sweep_once`` now and then every ``interval_sec`` until ``stop_event``.

    Extracted (and exception-hardened) so the timer behaviour is unit-testable
    without a real thread or a real clock: tests drive it with a tiny interval and a
    ``sweep_once`` that sets the stop event. A failing sweep never kills the loop.
    """
```

```
    while not stop_event.is_set():
        try:
            sweep_once()
        except Exception as e:  # noqa: BLE001 ? the sweeper must outlive any one pass
            logger.warning("[RETENTION] sweep pass failed (non-fatal): %s", e)
        stop_event.wait(interval_sec)


def start_retention_sweeper(
    config: Any, *, stop_event: Optional[threading.Event] = None
) -> Optional[threading.Thread]:
    """Server-boot entry point: start the periodic uploads sweep IF enabled.

    DEFAULT-OFF: with ``retention.sweep_enabled`` false/absent this returns ``None``
    and starts nothing ? zero behavior change. When enabled, a daemon thread sweeps
    ``security.upload_dir`` every ``retention.sweep_interval_hours`` (first pass
    immediately). Returns the thread (tests may pass ``stop_event`` to stop it).
    """
    retention = getattr(config, "retention", None)
    if retention is None or not getattr(retention, "sweep_enabled", False):
        logger.debug(
            "[RETENTION] in-server uploads sweep disabled "
            "(retention.sweep_enabled=false ? the default)."
        )
        return None

    security = getattr(config, "security", None)
    uploads_dir = getattr(security, "upload_dir", None) or "output/uploads"
    interval_hours = float(getattr(retention, "sweep_interval_hours", 24.0))
    retention_days = float(getattr(retention, "uploads_retention_days", 7.0))
    interval_sec = interval_hours * 3600.0

    logger.info(
        "[RETENTION] in-server uploads sweep ENABLED: dir=%s window=%gd interval=%gh "
        "(T11/#301; set retention.sweep_enabled=false to turn off)",
        uploads_dir,
        retention_days,
        interval_hours,
    )
    ev = stop_event if stop_event is not None else threading.Event()
    thread = threading.Thread(
        target=_sweep_loop,
        args=(ev, interval_sec, lambda: run_uploads_sweep(uploads_dir, retention_days)),
        name="uploads-retention-sweep",
        daemon=True,  # never blocks server shutdown
    )
    thread.start()
    return thread
```

36. user (2026-07-02T04:51:29.866Z)

```
File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-
sweep/backend/utils/retention.py (file state is current in your context ? no need to Read it
back)
```

37. user (2026-07-02T04:51:38.696Z)

```
3:# Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4-# (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5:# reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6-# auto-detection would otherwise miss the un-__init__'d `backend` root.
7-#
8-# torch/torchvision are deliberately NOT listed in [project.dependencies]:
9:# their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
```

```
10-# which PEP 621 cannot express. Install them separately ? see [project.optional-dependencies]
11-# `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12-[build-system]
13:requires = ["hatchling"]
14:build-backend = "hatchling.build"
15-
16-[project]
17-name = "badminton-highlight-indexer"
18-version = "0.1.0"
19-description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20-readme = "README.md"
21-requires-python = ">=3.10"
22-license = { text = "MIT" }
23-authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24-keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25-
26-# Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27-dependencies = [
28-    "fastapi>=0.111.0",
29-    "uvicorn>=0.30.1",
--
42:    # PyPI wheel is broken (metadata-only), so it MUST be installed from git ? see
43-    # requirements.txt. PEP 621 cannot express a git URL, so it's NOT listed here
44-    # (same pattern as torch/torchvision). CI installs it on its own line.
45-]
46-
47-[project.optional-dependencies]
48-# Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
49-dev = [
50-    "pytest",
51-    "pytest-cov",
52-    "httpx",        # fastapi TestClient transport
53-    "ruff",
54-    "mypy",
55-    "PyJWT[crypto]>=2.8",  # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
56-]
57-
--
72:# CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
73-# express, so they are NOT declarable here. Install them out-of-band instead:
74-#   # CUDA 12.4 box:
75-#   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
76-#   # CPU-only box (also what CI uses):
```

38. user (2026-07-02T04:51:53.171Z)

```
90-indexer = "backend.main:main"
91-
92:[tool.hatch.metadata]
93-# The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
94-# reference*); hatchling requires opting in to allow those in metadata.
95-allow-direct-references = true
96-
97:[tool.hatch.build.targets.wheel]
98-# `backend` is a namespace package (no __init__.py); list both import roots
99-# explicitly so an editable/wheel install exposes `import backend` and
100-# `import training` regardless of cwd.
101-packages = ["backend", "training"]
```

39. assistant (2026-07-02T04:52:45.066Z)

```
"""In-server uploads-zone retention sweep (T11 / #301 / audit R2-8).

The standalone tool (``python -m tools.cleanup_caches``) knows how to classify and
```

```
purge stale uploads, but nothing schedules it ? so on a hosted box the upload landing
zone (``security.upload_dir``) grows without bound and the 7-day privacy retention
window (owner policy, 2026-06-21) is unenforced. This module gives the server an
OPT-IN periodic sweep of the **uploads zone only** (the WSL stage dir and the
``output/`` cache zones stay with the tool / operator):

* ``retention.sweep_enabled`` (default **False**) ? nothing runs unless the owner
  flips it; the default server boot is byte-identical to before.
* ``retention.sweep_interval_hours`` (default 24) ? cadence of the daemon-thread
  timer; a sweep also runs once at startup so a rarely-restarted box still catches up.
* ``retention.uploads_retention_days`` (default 7) ? the T11 privacy window. Uploaded
  source videos and orphaned ``.partial-*`` files older than this are deleted.
  ? ``<= 0`` makes the sweep a NO-OP ? deliberately SAFER than the CLI tool, where a
  0 window means "no age guard" (sweep everything now).

The sweep REUSES the tool's classify/plan/scan/purge helpers by import (no shelling
out), so the decision logic stays single-sourced and unit-tested in one place. The
import is LAZY: the wheel packages only ``backend``/``training``, so ``tools`` exists
on a repo checkout (how hosted boxes run) but not necessarily in a pip install ? the
default-OFF path must never import it. Every purge is logged loudly ? one
``[RETENTION]`` line per deleted entry plus a summary ? so a hosted box's journal
shows exactly what was deleted and why (the observability working agreement).
Retention is time-window-only: uploads referenced by non-terminal jobs are NOT
exempted (#301's open design call ? owner to ratify a job-aware guard).

The systemd-timer alternative (running the standalone tool on a schedule instead of
enabling the in-server sweep) lives in ``deploy/systemd/``.
"""

from __future__ import annotations

import logging
import threading
import time
from typing import Any, Callable, Dict, Optional

logger = logging.getLogger(__name__)


def _cleanup_tool():
    """Lazily import ``tools.cleanup_caches`` (see the module docstring for why).

    Raises ImportError on a tools-less install ? callers surface it loudly rather
    than sweeping with re-implemented (divergence-prone) logic.
    """
    import tools.cleanup_caches as cleanup_caches

    return cleanup_caches


def run_uploads_sweep(
    uploads_dir: str, retention_days: float, *, now: Optional[float] = None
) -> Dict[str, Any]:
    """One T11 sweep pass over ``uploads_dir``: purge entries older than the window.

    Pure decision core reused from tools.cleanup_caches (classify via Entry.category
    inside ``plan_deletions``); ``now`` is injectable for fake-clock tests. Returns a
    summary dict (also logged) and never raises on individual deletion failures.
    """
    now = time.time() if now is None else now
    if retention_days <= 0:
        # An unattended daemon must never read "window off" as "purge everything now"
        # (the CLI's 0 = no-age-guard semantics). Refuse loudly instead.
        logger.warning(
            "[RETENTION] uploads_retention_days=%g is not a positive window ? "
```

```
            "skipping the sweep (nothing deleted).",
            retention_days,
        )
        return {"skipped": True, "purged": 0, "failed": 0, "kept": 0, "freed_bytes": 0}

    cc = _cleanup_tool()
    entries = cc.scan_uploads(uploads_dir)
    to_delete, kept = cc.plan_deletions(
        entries,
        keep_video=None,
        older_than_days=0.0,  # scan_uploads emits only location=="uploads" entries,
        include_wasbcache=False,  # so only the uploads window below applies.
        now=now,
        uploads_retention_days=retention_days,
    )
    freed = 0
    failed = 0
    for e in to_delete:
        age_days = (now - e.mtime) / 86400.0 if e.mtime > 0 else float("inf")
        if cc._delete(e, distro=""):  # location is never "wsl" here ? distro unused
            freed += e.size
            logger.info(
                "[RETENTION] PURGED %s (%s, %s, %.1fd old > %gd window)",
                e.path,
                e.category[0],
                cc.human(e.size),
                age_days,
                retention_days,
            )
        else:
            failed += 1
            logger.warning(
                "[RETENTION] FAILED to purge %s (%s) ? will retry next sweep",
                e.path,
                e.category[0],
            )
    summary: Dict[str, Any] = {
        "skipped": False,
        "purged": len(to_delete) - failed,
        "failed": failed,
        "kept": len(kept),
        "freed_bytes": freed,
    }
    logger.info(
        "[RETENTION] uploads sweep of %s: purged %d (%s freed), failed %d, kept %d "
        "(window %gd)",
        uploads_dir,
        summary["purged"],
        cc.human(freed),
        failed,
        summary["kept"],
        retention_days,
    )
    return summary


def _sweep_loop(
    stop_event: threading.Event, interval_sec: float, sweep_once: Callable[[], Any]
) -> None:
    """Run ``sweep_once`` now and then every ``interval_sec`` until ``stop_event``.

    Extracted (and exception-hardened) so the timer behaviour is unit-testable
    without a real thread or a real clock: tests drive it with a tiny interval and a
    ``sweep_once`` that sets the stop event. A failing sweep never kills the loop.
    """
```

```
    while not stop_event.is_set():
        try:
            sweep_once()
        except Exception as e:  # the sweeper must outlive any one failing pass
            logger.warning("[RETENTION] sweep pass failed (non-fatal): %s", e)
        stop_event.wait(interval_sec)


def start_retention_sweeper(
    config: Any, *, stop_event: Optional[threading.Event] = None
) -> Optional[threading.Thread]:
    """Server-boot entry point: start the periodic uploads sweep IF enabled.

    DEFAULT-OFF: with ``retention.sweep_enabled`` false/absent this returns ``None``
    and starts nothing ? zero behavior change. When enabled, a daemon thread sweeps
    ``security.upload_dir`` every ``retention.sweep_interval_hours`` (first pass
    immediately). Returns the thread (tests may pass ``stop_event`` to stop it).
    Never raises ? a sweeper that can't start must not take the server down with it.
    """
    retention = getattr(config, "retention", None)
    if retention is None or not getattr(retention, "sweep_enabled", False):
        logger.debug(
            "[RETENTION] in-server uploads sweep disabled "
            "(retention.sweep_enabled=false ? the default)."
        )
        return None

    try:
        _cleanup_tool()  # fail LOUDLY at boot, not silently every interval
    except ImportError as e:
        logger.error(
            "[RETENTION] retention.sweep_enabled=true but tools.cleanup_caches is not "
            "importable (%s) ? the sweep needs a repo checkout (the wheel does not "
            "ship tools/). NOT sweeping; use the deploy/systemd/ timer instead.",
            e,
        )
        return None

    security = getattr(config, "security", None)
    uploads_dir = getattr(security, "upload_dir", None) or "output/uploads"
    interval_hours = float(getattr(retention, "sweep_interval_hours", 24.0))
    retention_days = float(getattr(retention, "uploads_retention_days", 7.0))
    interval_sec = interval_hours * 3600.0

    logger.info(
        "[RETENTION] in-server uploads sweep ENABLED: dir=%s window=%gd interval=%gh "
        "(T11/#301; set retention.sweep_enabled=false to turn off)",
        uploads_dir,
        retention_days,
        interval_hours,
    )
    ev = stop_event if stop_event is not None else threading.Event()
    thread = threading.Thread(
        target=_sweep_loop,
        args=(ev, interval_sec, lambda: run_uploads_sweep(uploads_dir, retention_days)),
        name="uploads-retention-sweep",
        daemon=True,  # never blocks server shutdown
    )
    thread.start()
    return thread
```

40. user (2026-07-02T04:52:45.094Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-
```

sweep/backend/utils/retention.py has been updated successfully. (file state is current in your context ? no need to Read it back)

41. assistant (2026-07-02T04:52:49.559Z)

Now wire it into the server boot path in main.py.

42. user (2026-07-02T04:52:50.866Z)

```
76      )
77      from backend.utils.freespace import require_free_bytes  # HTTP-507 disk guard (P2, D11)
78      from backend.utils.hashing import compute_video_id  # canonical file-identity hashing
79      from backend.utils.paths import (
80          output_base,
81          resolve_under_root,
82          safe_output_filename,
83          validate_input_path,
84      )
85      from backend.utils.redact import (
86          redact_secrets,  # P0b: mask secret-shaped fields in /api/config
87      )
88      from backend.utils.single_instance import acquire_lock  # P0e: one instance per database
89
90      app = FastAPI(title="Sports Highlight Indexer API")
91
```

43. user (2026-07-02T04:52:59.604Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-sweep/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

44. user (2026-07-02T04:53:09.320Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-sweep/backend/main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

45. assistant (2026-07-02T04:53:14.061Z)

Now the tool docstring update.

46. user (2026-07-02T04:53:22.392Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

47. user (2026-07-02T04:53:30.434Z)

```
1       """Cache-hygiene + retention sweep for the GPU highlight pipeline.
2
3       Scans the regenerable intermediates the pipeline leaves behind ? in the WSL stage
4       dir (``~/clips``), the Windows ``output/`` tree, and the upload landing zone
5       (``security.upload_dir``, default ``output/uploads``) ? classifies each entry as
6       **purge** (regenerable: frame caches, staged video copies, handoff clips, temp dirs;
7       and, past a retention window, uploaded source videos) or **keep** (durable: trajectory
8       CSVs, the SQLite DB, final reels) and reports the reclaimable space.
9
10      **Retention sweep (T11 ? pre-alpha-exposure):** uploaded source videos in the upload
11      dir are a *privacy*-driven purge, not a *disk*-driven one ? once a tester's footage is
12      older than ``--uploads-retention-days`` (default 7) it is swept, along with the orphaned
13      ``.partial-<id>`` temp files aborted uploads leave behind. Reels / DB / CSVs stay
durable
14      per the cache-hygiene policy (the owner's call, 2026-06-21).
15
```

```
16      DRY-RUN BY DEFAULT ? it only *shows* what it would delete. Pass ``--apply`` to delete.
17
18          python -m tools.cleanup_caches                      # report only (safe)
19          python -m tools.cleanup_caches --apply              # delete purge candidates
20          python -m tools.cleanup_caches --keep-video 89605e80ed01   # protect one video's
cache
21          python -m tools.cleanup_caches --older-than 7       # only purge regenerable caches
>7d old
22          python -m tools.cleanup_caches --uploads-retention-days 3   # sweep uploads >3d old
23          python -m tools.cleanup_caches --include-wasbcache  # also drop detector resume
caches
24          python -m tools.cleanup_caches --json               # machine-readable plan (for a
cron/job wrapper)
25
26      Pairs with the pipeline's own self-cleaning (indexing.{wasb,tracknet}.keep_frames=false,
27      which deletes frames after a SUCCESSFUL run); this tool reclaims what failed/old runs
28      and pre-self-cleaning videos left behind, and enforces the upload retention window
before
29      non-owner (alpha) exposure. Stdlib only; safe to run anywhere (the WSL scan is skipped
30      automatically when ``wsl`` is unavailable). ``--uploads-dir`` must match
``security.upload_dir``
31      (its default matches the config default, ``output/uploads``).
32      """
33
34      from __future__ import annotations
35
```

48. user (2026-07-02T04:53:44.333Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-
sweep/tools/cleanup_caches.py has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

49. assistant (2026-07-02T04:53:51.332Z)

```
Now the systemd deploy kit.
```

50. assistant (2026-07-02T04:54:01.847Z)

T11 uploads retention sweep (#301 / audit R2-8) ? oneshot unit, driven by khelsutra-retention.timer (install BOTH files; see deploy/systemd/README.md).

#

Runs the standalone tool with --apply so tester footage older than the privacy window is deleted and orphaned .partial-* files are reclaimed. Every PURGED line lands in journald (StandardOutput=journal) for the audit trail.

#

ADJUST per box before installing:

- **User / WorkingDirectory / the venv python path ? match the engine service (e.g. khelsutra-engine.service on the droplet).**
- **--uploads-dir MUST match the server's security.upload_dir (droplet runbook default: /srv/khelsutra/incoming/uploads).**
- **--output-dir points at the server's output dir; --older-than 7 keeps the regenerable-cache side age-guarded so a fresh in-flight run's intermediates are never yanked out from under it. (Drop the pair if you want uploads-only:**

point --output-dir at a directory that doesn't exist.)

#

NOTE: do NOT add --json here ? --json is a report-only plan view; the tool returns before deleting anything even with --apply.

#

Alternative: skip this unit entirely and set retention.sweep_enabled=true in the server's config.json (the in-server daemon sweep; backend/utils/retention.py). Pick ONE scheduler so the journal has a single audit trail.

```
[Unit]
Description=Khelsutra uploads retention sweep (T11 privacy window)
Documentation=https://github.com/Khelsutra/badminton-highlight-
indexer/blob/master/tools/cleanup_caches.py

[Service]
Type=oneshot
User=khelsutra
WorkingDirectory=/srv/khelsutra/badminton-highlight-indexer
ExecStart=/srv/khelsutra/venv/bin/python -m tools.cleanup_caches --apply \
    --uploads-retention-days 7 \
    --uploads-dir /srv/khelsutra/incoming/uploads \
    --output-dir /srv/khelsutra/output \
    --older-than 7
StandardOutput=journal
StandardError=journal
```

51. user (2026-07-02T04:54:01.907Z)

```
File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-
sweep/deploy/systemd/khelsutra-retention.service (file state is current in your context ? no
need to Read it back)
```

52. assistant (2026-07-02T04:54:04.931Z)

Daily trigger for khelsutra-retention.service (T11 uploads retention sweep, #301). Persistent=true ? a missed window (box was off/rebooting) fires on the next boot, so the 7-day privacy retention can't silently lapse across downtime.

```
[Unit]
Description=Daily Khelsutra uploads retention sweep (T11, #301)

[Timer]
OnCalendar=daily
RandomizedDelaySec=1h
Persistent=true

[Install]
WantedBy=timers.target
```

53. user (2026-07-02T04:54:04.965Z)

```
File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-
sweep/deploy/systemd/khelsutra-retention.timer (file state is current in your context ? no need
to Read it back)
```

54. assistant (2026-07-02T04:54:28.106Z)

Uploads retention sweep ? scheduling kit (T11 / #301 / audit R2-8)

The engine ships a retention sweep (`python -m tools.cleanup_caches`,
`--uploads-retention-days` default 7) that enforces the 7-day privacy window on the
upload landing zone (`security.upload_dir`) ? but **nothing schedules it by default**.
On a hosted box you must pick ONE of the two supported schedulers below; a local
single-user install needs neither (nothing is ever deleted by default).

Option A ? in-server periodic sweep (no systemd needed)

Set in the serving box's `config.json`:

```json
{
  "retention": {
    "sweep_enabled": true,
    "sweep_interval_hours": 24,
    "uploads_retention_days": 7
  }
}
```

On the next server start a daemon thread sweeps `security.upload_dir` immediately and
then every `sweep_interval_hours`, logging one `[RETENTION] PURGED ?` line per deleted
entry (see `backend/utils/retention.py`). **DEFAULT-OFF** ? with `sweep_enabled=false`
(or the section absent) the server is byte-identical to before this knob existed.
Scope: the uploads zone ONLY; WSL stage-dir / `output/` cache cleanup stays with the
tool (Option B covers both).

Option B ? systemd timer running the standalone tool

Install the two units in this directory (edit the paths/user in the `.service` first ?
they must match your engine service, e.g. `khelsutra-engine.service` on the droplet,
and `--uploads-dir` must match the server's `security.upload_dir`):

```bash
sudo cp deploy/systemd/khelsutra-retention.{service,timer} /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now khelsutra-retention.timer
```

Verify:

```bash
systemctl list-timers khelsutra-retention.timer   # next/last trigger
sudo systemctl start khelsutra-retention.service  # force one sweep now
journalctl -u khelsutra-retention.service -n 50   # the PURGED/KEEP audit trail
```

No systemd? The equivalent cron line (same caveats as the `.service` header):

```
17 3 * * * cd /srv/khelsutra/badminton-highlight-indexer && /srv/khelsutra/venv/bin/python -m
tools.cleanup_caches --apply --uploads-retention-days 7 --uploads-dir
/srv/khelsutra/incoming/uploads --output-dir /srv/khelsutra/output --older-than 7 >>
/var/log/khelsutra-retention.log 2>&1
```

Which one?

* **Option A** ? simplest for a box that runs the server 24/7 (the droplet alpha);
  survives nothing extra, one config flip, journald already captures server logs.
* **Option B** ? sweeps even when the server is down/crashed, and also reclaims the

```
  regenerable cache zones; use it where an ops-grade, server-independent guarantee
  matters. `Persistent=true` back-fills a missed window after downtime.

Don't enable both ? double-scheduling is harmless (the second sweep finds nothing)
but splits the audit trail across two journals.

Design note (#301, open): retention is time-window-only ? an upload still referenced
by a queued/running job row is NOT exempted. The window (7d) is far longer than any
job's life, so this is theoretical; a job-aware guard is the owner's call.
```

55. user (2026-07-02T04:54:28.140Z)

```
File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-
sweep/deploy/systemd/README.md (file state is current in your context ? no need to Read it back)
```

56. user (2026-07-02T04:54:36.240Z)

```
(Bash completed with no output)
```

57. user (2026-07-02T04:54:37.254Z)

```
55      ## Gotchas
56      - The dispatched container **must run inline** ? `CloudRunCompute` forces
`compute_target=local_gpu`
57        + `indexing.detector_impl=native` in the per-execution args so the Job never re-
dispatches itself.
58      - Completion is a **bucket marker** (`<out_uri>/_dispatch/<token>.done`), not the Cloud
Run API status
59        ? so it reuses the storage layer + `execution_policy.poll_until` and survives a
control-plane restart.
60      - A **failed** Job (validation rc 2 / segmenter rc 3) now writes a **failure marker**
carrying the
61        real non-zero rc, so the dispatcher's poll terminates **FAST + accurately** (no 30-min
hang on a bad
62        video). Only a **hung/crashed** container (no marker at all) makes the bounded poll
time out
63        (`ExecutionPolicy.max_wait_sec`); the worker CLI maps that to a clean **rc 4**, and
the API path
64        surfaces it as a `cloud dispatch error` job failure (not a traceback).
65      - libx264 encode (parity, D16) + NVDEC decode-only; the image keeps ffmpeg's default
libx264.
66      - **`GOOGLE_CLOUD_PROJECT` is required** for the real client ?
`GoogleCloudRunJobClient.run_job`
67        fails fast with a clear `RuntimeError` if it is unset/empty (it must resolve a real
`job_path`;
68        an unset project would otherwise build a bogus `projects/None/...` name). Export it
(step 3) before
69        dispatching. The guard runs before the `google-cloud-run` import, so it is exercised
in CI even
70        though the SDK isn't a CI/test dependency.
71
```

58. user (2026-07-02T04:54:49.995Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-
sweep/deploy/cloudrun/README.md has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

59. assistant (2026-07-02T04:55:08.486Z)

Now the tests.

60. assistant (2026-07-02T04:55:39.362Z)

```
"""Tests for the in-server uploads retention sweep (T11 / #301 / audit R2-8).

Covers the three contract points of backend/utils/retention.py:
  * the timer RESPECTS the flag ? default-OFF starts nothing (zero behavior change);
  * the sweep pass itself, on a real tmpdir with a FAKE clock (mtimes pinned via
    os.utime against an injected `now`) ? old uploads purged, fresh/unknown kept;
  * the loop plumbing ? first pass runs immediately, a failing pass never kills the
    loop, and the stop event ends it.
The classification/window rules themselves are the tool's and stay unit-tested in
tests/test_cleanup_caches.py ? these tests guard the server-side reuse of them.
"""

import logging
import os
import threading

from backend.config.models import AppConfig, RetentionConfig
from backend.utils import retention

FAKE_NOW = 2_000_000_000.0  # fixed fake epoch ? tests never read the real clock


def _mk_file(path, age_days, size=5):
    path.write_bytes(b"x" * size)
    ts = FAKE_NOW - age_days * 86400.0
    os.utime(path, (ts, ts))
```

---- config knobs ----------

```
def test_retention_config_defaults_are_off_and_safe():
    cfg = AppConfig()
    assert cfg.retention.sweep_enabled is False  # DEFAULT-OFF (R2-8 zero-change bar)
    assert cfg.retention.sweep_interval_hours == 24.0
    assert cfg.retention.uploads_retention_days == 7.0


def test_retention_config_accepts_overrides():
    cfg = AppConfig(retention={"sweep_enabled": True, "sweep_interval_hours": 6})
    assert cfg.retention.sweep_enabled is True
    assert cfg.retention.sweep_interval_hours == 6.0
```

---- the timer respects the flag ----------

```
def test_sweeper_not_started_when_disabled_by_default():
    assert retention.start_retention_sweeper(AppConfig()) is None


def test_sweeper_not_started_when_config_has_no_retention_section():
    assert retention.start_retention_sweeper(object()) is None


def test_sweeper_starts_daemon_thread_and_sweeps_configured_dir(tmp_path, monkeypatch):
    calls = []
    ev = threading.Event()

    def fake_sweep(uploads_dir, retention_days, **kwargs):
        calls.append((uploads_dir, retention_days))
```

```
        ev.set()  # end the loop after the first (immediate) pass

    monkeypatch.setattr(retention, "run_uploads_sweep", fake_sweep)
    cfg = AppConfig(
        retention={"sweep_enabled": True, "uploads_retention_days": 3},
        security={"upload_dir": str(tmp_path)},
    )
    thread = retention.start_retention_sweeper(cfg, stop_event=ev)
    assert thread is not None
    assert thread.daemon is True  # must never block server shutdown
    thread.join(timeout=10)
    assert not thread.is_alive()
    assert calls == [(str(tmp_path), 3.0)]  # security.upload_dir + retention window
```

---- the sweep pass (fake clock + tmpdir) ----------

```
def test_sweep_purges_old_uploads_keeps_fresh_and_unknown(tmp_path, caplog):
    _mk_file(tmp_path / "stale.mp4", age_days=9)  # past the 7d window -> purged
    _mk_file(tmp_path / ".partial-orphan", age_days=9)  # old orphan -> purged
    _mk_file(tmp_path / "fresh.mp4", age_days=2)  # inside the window -> kept
    _mk_file(tmp_path / ".partial-live", age_days=0.001)  # in-flight -> kept
    _mk_file(tmp_path / "notes.txt", age_days=30)  # unknown category -> kept
    subdir = tmp_path / "user_42"  # future PR-3 per-user dir -> untouched
    subdir.mkdir()
    os.utime(subdir, (FAKE_NOW - 30 * 86400, FAKE_NOW - 30 * 86400))

    with caplog.at_level(logging.INFO, logger="backend.utils.retention"):
        summary = retention.run_uploads_sweep(str(tmp_path), 7.0, now=FAKE_NOW)

    assert not (tmp_path / "stale.mp4").exists()
    assert not (tmp_path / ".partial-orphan").exists()
    assert (tmp_path / "fresh.mp4").exists()
    assert (tmp_path / ".partial-live").exists()
    assert (tmp_path / "notes.txt").exists()
    assert subdir.exists()
    assert summary["purged"] == 2 and summary["failed"] == 0
    assert summary["kept"] == 4 and summary["freed_bytes"] == 10
    # Loud observability: one PURGED line per deleted entry + a summary line.
    purged_lines = [m for m in caplog.messages if "PURGED" in m]
    assert len(purged_lines) == 2
    assert any("uploads sweep of" in m for m in caplog.messages)


def test_sweep_nonpositive_window_is_a_noop(tmp_path, caplog):
    # Unlike the CLI (0 = no age guard = sweep everything), the unattended in-server
    # sweep must refuse a non-positive window rather than purge every upload.
    _mk_file(tmp_path / "ancient.mp4", age_days=100)
    with caplog.at_level(logging.WARNING, logger="backend.utils.retention"):
        summary = retention.run_uploads_sweep(str(tmp_path), 0.0, now=FAKE_NOW)
    assert (tmp_path / "ancient.mp4").exists()
    assert summary == {
        "skipped": True,
        "purged": 0,
        "failed": 0,
        "kept": 0,
        "freed_bytes": 0,
    }
    assert any("skipping the sweep" in m for m in caplog.messages)


def test_sweep_missing_dir_is_harmless(tmp_path):
    summary = retention.run_uploads_sweep(str(tmp_path / "nope"), 7.0, now=FAKE_NOW)
```

```
    assert summary["purged"] == 0 and summary["failed"] == 0


def test_sweep_default_config_window_matches_the_tool_default():
    # The typed default (7d) must match the tool CLI's --uploads-retention-days
    # default so the two supported schedulers enforce the SAME privacy window.
    import tools.cleanup_caches as cc

    ap_default = None
    for action in cc.main.__code__.co_consts:  # fallback if argparse introspection fails
        pass
    import argparse

    parser = argparse.ArgumentParser()
    # Parse the tool's default the honest way: run its main() argv-parsing path via
    # a --json dry call would do I/O; instead read the declared default directly.
    ns = None
    del parser, ns, ap_default, action
    assert RetentionConfig().uploads_retention_days == 7.0
```

---- the loop plumbing ----------

```
def test_sweep_loop_runs_immediately_then_respects_stop_event():
    ev = threading.Event()
    count = {"n": 0}

    def sweep_once():
        count["n"] += 1
        if count["n"] >= 3:
            ev.set()

    retention._sweep_loop(ev, interval_sec=0.001, sweep_once=sweep_once)
    assert count["n"] == 3  # first pass immediate, loop ends when the event is set


def test_sweep_loop_survives_a_failing_pass(caplog):
    ev = threading.Event()
    count = {"n": 0}

    def sweep_once():
        count["n"] += 1
        if count["n"] == 1:
            raise RuntimeError("disk went away")
        ev.set()

    with caplog.at_level(logging.WARNING, logger="backend.utils.retention"):
        retention._sweep_loop(ev, interval_sec=0.001, sweep_once=sweep_once)
    assert count["n"] == 2  # the failure did not kill the loop
    assert any("non-fatal" in m for m in caplog.messages)
```

61. user (2026-07-02T04:55:39.393Z)

```
File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-
badminton-highlight-indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-
sweep/tests/test_retention_sweep.py (file state is current in your context ? no need to Read it
back)
```

62. assistant (2026-07-02T04:55:53.153Z)

```
I left a half-written parity test in there ? let me fix it properly.
```

63. user (2026-07-02T04:55:59.980Z)

```
The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-
indexer/c1fcb4aa-537d-41fd-858e-53f93349bf5a/scratchpad/wt-retention-
sweep/tests/test_retention_sweep.py has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

64. assistant (2026-07-02T04:56:05.600Z)

Now run the targeted tests.

65. user (2026-07-02T04:56:12.422Z)

```
.........................................                                [100%]
44 passed in 3.29s
```

66. user (2026-07-02T04:56:54.292Z)

```
All checks passed!
backend\sports\__init__.py:9: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs  [annotation-unchecked]
backend\providers\__init__.py:9: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs  [annotation-unchecked]
Success: no issues found in 3 source files
```

67. user (2026-07-02T04:57:36.600Z)

```
[check-md-links] clean ? no broken internal links in live docs.
  C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
    warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
99 passed, 1 skipped, 3 warnings in 27.97s
```

68. assistant (2026-07-02T04:57:46.208Z)

All green. Now stage explicit paths and verify branch state before committing.

69. user (2026-07-02T04:57:49.343Z)

```
 M backend/config/models.py
 M backend/main.py
 M deploy/cloudrun/README.md
 M tools/cleanup_caches.py
?? backend/utils/retention.py
?? deploy/systemd/
?? tests/test_retention_sweep.py
r2-8-retention-sweep-scheduler
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
```

70. user (2026-07-02T04:57:58.599Z)

```
warning: in the working copy of 'backend/utils/retention.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'deploy/systemd/README.md', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'deploy/systemd/khelsutra-retention.service', LF will be
replaced by CRLF the next time Git touches it
warning: in the working copy of 'deploy/systemd/khelsutra-retention.timer', LF will be replaced
by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_retention_sweep.py', LF will be replaced by CRLF the
next time Git touches it
```

```
M  backend/config/models.py
M  backend/main.py
A  backend/utils/retention.py
M  deploy/cloudrun/README.md
A  deploy/systemd/README.md
A  deploy/systemd/khelsutra-retention.service
A  deploy/systemd/khelsutra-retention.timer
A  tests/test_retention_sweep.py
M  tools/cleanup_caches.py
```

71. user (2026-07-02T04:58:14.011Z)

```
r2-8-retention-sweep-scheduler
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
[r2-8-retention-sweep-scheduler 36576c4] feat(retention): schedule the T11 uploads sweep ? opt-
in in-server timer + systemd deploy kit (#301, audit R2-8)
 9 files changed, 545 insertions(+)
 create mode 100644 backend/utils/retention.py
 create mode 100644 deploy/systemd/README.md
 create mode 100644 deploy/systemd/khelsutra-retention.service
 create mode 100644 deploy/systemd/khelsutra-retention.timer
 create mode 100644 tests/test_retention_sweep.py
```

72. assistant (2026-07-02T04:58:14.520Z)

```
You've hit your session limit · resets 1:10pm (Asia/Calcutta)
```